

HTTP- & time-triggered Azure Functions

In this lecture we will learn how to create a **REST API** using Azure functions. **Azure Functions** is a serverless computing service in Microsoft Azure, allowing you to run event-triggered functions without managing infrastructure. When building RESTful APIs with Azure Functions, you create functions that respond to HTTP requests. REST API, which stands for Representational State Transfer Application Programming Interface, is a set of rules and conventions for building and interacting with web services. It is an architectural style that uses a stateless communication model, meaning each request from a client contains all the information needed to understand and process the request. RESTful APIs are designed to be simple, scalable, and stateless, making them well-suited for the web.

Key characteristics of RESTful APIs include:

1. **Statelessness:** Each request from a client to a server must contain all the information needed to understand and process the request. The server should not store any information about the client's state between requests.
2. **Resources:** Resources, such as data objects or services, are identified by URLs. Clients interact with these resources using standard HTTP methods like GET, POST, PUT, and DELETE.
3. **Representations:** Resources can have multiple representations, such as JSON or XML. Clients and servers negotiate the representation format during interactions.
4. **Uniform Interface:** A uniform and consistent set of conventions for interacting with resources, including standard methods and conventions for resource identification.
5. **Stateless Communication:** Each request from a client to a server must contain all the information needed to understand and process the request.

For example, in a RESTful API for a blog, a client might use the HTTP GET method to retrieve a list of blog posts from a server at the URL `"/api/posts"`. The server would respond with a representation of the blog posts, typically in JSON or XML format. RESTful APIs are widely used in web development for building web services that can be easily consumed by various clients, such as web browsers or mobile applications. They provide a standardized way for different systems to communicate over the internet.

1. Introduction to Azure Functions

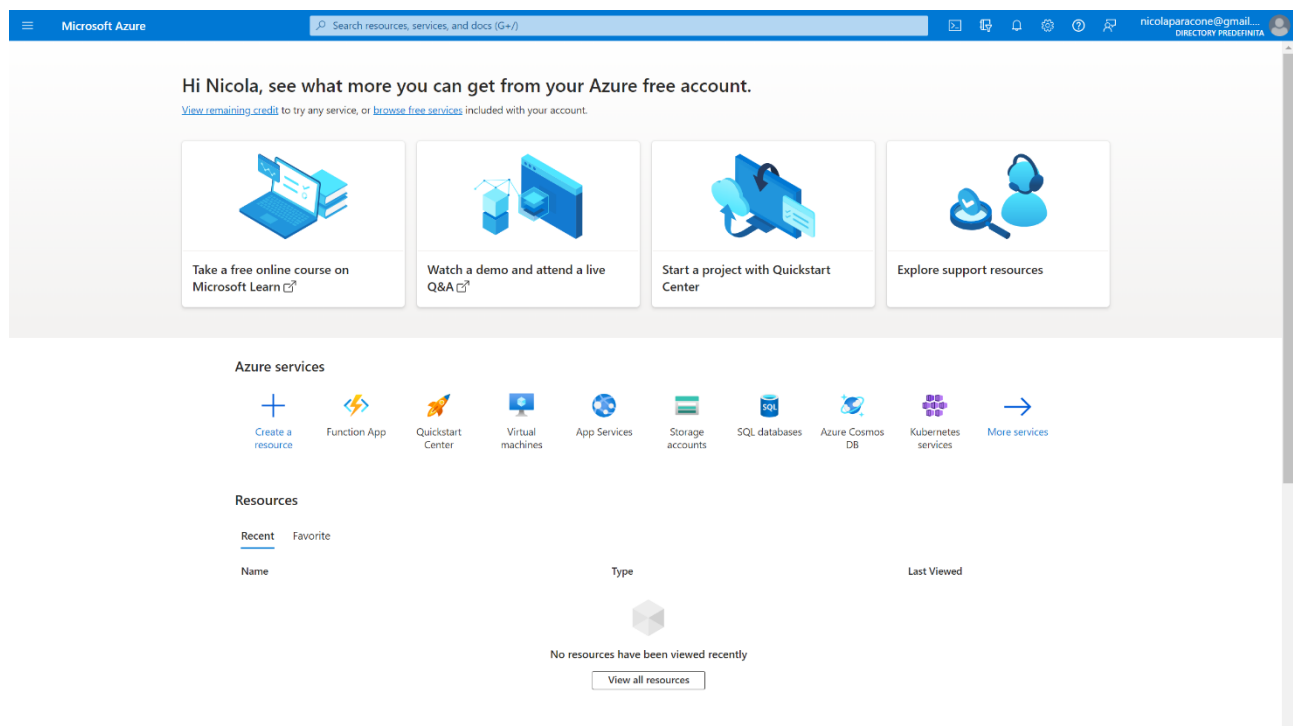
Azure Functions is a serverless compute service offered by Microsoft Azure, designed for developers to write code without managing the underlying infrastructure. In this model, functions are individual units of code that can be triggered by various events, such as HTTP requests, changes in data, or updates in storage. One notable feature is its pay-as-you-go model, where users only pay for the actual compute resources consumed during function execution, eliminating the need for pre-allocated or reserved resources. Azure Functions supports multiple programming languages, including C#, JavaScript (Node.js), Python, and PowerShell, giving developers flexibility in their language choice. Scalability is a key advantage, as Azure Functions automatically scales based on demand. It seamlessly adjusts resources to handle increased workloads and scales down during periods of decreased demand. Additionally, Azure Functions easily integrates with other Azure services and third-party tools, facilitating the creation of workflows and connections between different components. The stateless nature of functions is a design principle, meaning they don't retain information between executions. This characteristic is well-suited for event-driven workloads. Azure Functions supports various developer tools, including Visual Studio, Visual Studio Code, and Azure Functions Core Tools. Developers commonly leverage Azure Functions for tasks such as data processing, responding to HTTP requests, integrating with Azure services, and implementing serverless architectures.

The focus on simplicity, event-driven triggers, and seamless integration makes Azure Functions a valuable tool for developers building scalable and responsive applications in a serverless environment.

Azure Functions in Dynamics 365 allow you to extend and customize the platform in various ways. You can create custom workflows and business logic tailored to your organization's specific needs, going beyond the capabilities provided by standard Dynamics 365 workflows. These functions are adept at handling events triggered by actions within Dynamics 365, such as record creation, updates, or deletions. This real-time event handling capability empowers you to execute custom code promptly in response to changes within the Dynamics 365 environment. When it comes to data integration, Azure Functions serve as a valuable tool for seamlessly connecting Dynamics 365 with other systems or services. This integration proves useful in scenarios where data synchronization or transfer between Dynamics 365 and external databases, applications, or APIs is necessary. Developers can also create HTTP-triggered Azure Functions to act as custom APIs, facilitating the interaction between external systems and Dynamics 365. This capability opens the door to integrating with third-party applications, services, or building custom front-end interfaces. Automation of repetitive tasks is another strength of Azure Functions in Dynamics 365. By triggering functions through scheduled events, you can automate routine operations such as data cleanup, validation, and other tasks, reducing manual effort and improving overall efficiency. Azure Functions also offer a platform for implementing complex business logic that requires additional computational power or specialized processing. This logic can be seamlessly integrated into Dynamics 365, enhancing the platform's capabilities to meet the specific requirements of your organization. In essence, Azure Functions provide a flexible and scalable means to extend the functionality of Dynamics 365, enabling you to tailor the platform to your organization's unique needs and seamlessly integrate it with other systems and services.

2. Creating an Azure function in Azure

First, let us connect to the Azure portal (<https://portal.azure.com/#home>). Here we are using a trial subscription to Azure.



Click on Function App.

Microsoft Azure Upgrade Search resources, services, and docs (G+/I)

Home > Function App Directory predefinita

+ Create Manage view Refresh Export to CSV Open query Assign tags Start Restart Stop Delete

Filter for any field... Subscription equals all Resource group equals all Location equals all Add filter

Showing 0 to 0 of 0 records.

No grouping List view

Name Status Location Pricing Tier App Service Plan Subscription App Type

No function apps match your filters

Try changing or clearing your filters.

Clear filters

Learn more about App Service

Give feedback

Click on + Create.

Microsoft Azure Upgrade Search resources, services, and docs (G+/I)

Home > Function App > Create Function App

Basics Networking Monitoring Deployment tags Review + create

Create a function app, which lets you group functions as a logical unit for easier management, deployment and sharing of resources. Functions lets you execute your code in a serverless environment without having to first create a VM or publish a web application.

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription Azure subscription 1

Resource Group (New) Resource group Create new

Instance Details

Function App name Function App name.azurewebsites.net

Do you want to deploy code or container image? Code Container Image

Runtime stack Select a runtime stack

Version Select a runtime stack version

Region East US

Operating system

The Operating System has been recommended for you based on your selection of runtime stack.

Operating System Linux Windows

Hosting

The plan you choose dictates how your app scales, what features are enabled, and how it is priced. Learn more

Review + create Previous Next: Networking

Here we must choose a valid Subscription, a Resource Group. A **resource group** is a logical container used to organize and manage related Azure resources. Resources within a resource group are typically deployed, managed, and monitored as a group, simplifying the management and organization of various Azure services. Resource groups provide a way to manage and apply policies, permissions, and tags collectively to the resources they contain. They also enable us to deploy, update, or delete all resources within the group as a single unit. This organizational structure enhances the overall efficiency of managing resources in Azure, especially when dealing with complex solutions comprising multiple services.

Let us create a new resource group called ExpenseGroup_Demo. Next we proceed by choosing a name for the function app, for example FunctionExpenseDemo. Since we want to create an Azure function using code, we select the .NET runtime stack and an appropriate version of this. The other fields are compiled as in the following figure.

Microsoft Azure Upgrade Search resources, services, and docs (G+/I)

Home > Function App >

Create Function App

Basics Networking Monitoring Deployment Tags Review + create

Create a function app, which lets you group functions as a logical unit for easier management, deployment and sharing of resources. Functions lets you execute your code in a serverless environment without having to first create a VM or publish a web application.

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * Azure subscription 1

Resource Group * (New) ExpenseGroup_Demo
[Create new](#)

Instance Details

Function App name * FunctionExpenseDemo ✓

Do you want to deploy code or container image? * ☒ Code ☐ Container Image

Runtime stack * .NET

Version * 6 (LTS)

Region * West Europe

Operating system

The Operating System has been recommended for you based on your selection of runtime stack.

Operating System * ☐ Linux ☒ Windows

Hosting

The plan you choose dictates how your app scales, what features are enabled, and how it is priced. [Learn more](#)

Hosting options and plans * ☒ Consumption (Serverless)
Optimized for serverless and event-driven workloads.

☐ Functions Premium (Not supported for selected region and operating system)
Event based scaling and network isolation, ideal for workloads running continuously.

☐ App service plan (Not supported for selected subscription)
Fully isolated and dedicated environment suitable for workloads that need large SKUs or need to co-locate Web Apps and Functions.

[Review + create](#) < Previous Next: Networking >

Consumption is a good choice for development and testing, but when it comes to production it is recommended to choose Functions Premium.

In the next step, we should select a **storage account**, which is a core Azure service that provides a scalable and secure location to store and manage various types of data. It serves as a central repository for storing and retrieving data used by Azure services, applications, and virtual machines. For example, it can store log files. In the case of a trial Azure subscription this tag does not appear, so that we will need to manually associate a storage account to our function app (we will do this later).

For Networking, Monitoring, Deployment and Tags we can leave the default options.

The following is a summary of the options we have selected.

The screenshot shows the 'Create Function App' wizard in the Microsoft Azure portal, specifically the 'Review + create' tab. The interface includes a top navigation bar with the Microsoft Azure logo, an 'Upgrade' button, and a search bar. Below the navigation bar, the breadcrumb trail shows 'Home > Function App >'. The main content area is divided into sections: 'Summary', 'Details', 'Hosting', 'Monitoring (New)', and 'Deployment'. The 'Details' section shows the Subscription (2305b9dc-c7f8-4afd-bcdf-21cf897a7188), Resource Group (ExpenseGroup_Demo), Name (FunctionExpenseDemo), and Runtime stack (.NET 6 (LTS)). The 'Hosting' section shows the Plan (New), Hosting options and plans (Consumption (Serverless)), Name (ASP-ExpenseGroupDemo-aa03), Operating System (Windows), Region (West Europe), and SKU (Dynamic). The 'Monitoring (New)' section shows Application Insights (Enabled), Name (FunctionExpenseDemo), and Region (West Europe). The 'Deployment' section shows Continuous deployment (Not enabled / Set up after app creation). At the bottom, there are buttons for 'Create', '< Previous', 'Next >', and 'Download a template for automation'.

Section	Property	Value
Details	Subscription	2305b9dc-c7f8-4afd-bcdf-21cf897a7188
	Resource Group	ExpenseGroup_Demo
	Name	FunctionExpenseDemo
	Runtime stack	.NET 6 (LTS)
Hosting	Plan (New)	
	Hosting options and plans	Consumption (Serverless)
	Name	ASP-ExpenseGroupDemo-aa03
	Operating System	Windows
	Region	West Europe
	SKU	Dynamic
Monitoring (New)	Application Insights	Enabled
	Name	FunctionExpenseDemo
	Region	West Europe
Deployment	Continuous deployment	Not enabled / Set up after app creation

Once you click on Create, it takes 2 to 3 minutes for completing the activation of the new function app.

The screenshot shows the 'FunctionExpenseDemo' overview page in the Microsoft Azure portal. The top navigation bar includes the Microsoft Azure logo, an 'Upgrade' button, and a search bar. Below the navigation bar, the breadcrumb trail shows 'Home >'. The main content area is divided into sections: 'Overview', 'Essentials', 'Functions', 'Deployment', and 'Settings'. The 'Overview' section shows a warning message: 'Storage is not configured properly. Function scaling will be limited. Click to learn more.' The 'Essentials' section shows the Resource group (ExpenseGroup_Demo), Status (Running), Location (West Europe), Subscription (Azure subscription 1), and Subscription ID (2305b9dc-c7f8-4afd-bcdf-21cf897a7188). The 'Functions' section shows a table with columns for Functions, Metrics, Properties, and Notifications (1). The 'Deployment' section shows Deployment slots and Deployment Center. The 'Settings' section shows Configuration, Authentication, Application Insights, Identity, Backups, Custom domains, Certificates, and Networking. At the bottom, there is a section titled 'Create functions in your preferred environment' with three options: 'Create in Azure portal', 'VS Code Desktop', and 'Other editors or CLI'. Each option has a 'Create function' button.

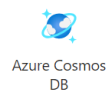
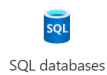
Property	Value
Resource group (move)	ExpenseGroup_Demo
Status	Running
Location (move)	West Europe
Subscription (move)	Azure subscription 1
Subscription ID	2305b9dc-c7f8-4afd-bcdf-21cf897a7188
Tags (edit)	

Create functions in your preferred environment

- Create in Azure portal**
Best optimized for:
 - Getting started without local setup
 - Choose from our Function templates[Create function](#)
- VS Code Desktop**
Best optimized for:
 - Local development within VS Code
 - Custom development tool requirements[Create with VS Code Desktop](#)
- Other editors or CLI**
Best optimized for:
 - Using preferred editor for development
 - Visual Studio, IntelliJ, command line[Set up your editor](#)

Notice the warning “Storage is not configured properly, Function scaling will be limited. Click to learn more”. To solve this issue, we start by creating a storage account. Go back to Home and click on Storage accounts.

Azure services



Microsoft Azure Upgrade Search resources, services, and docs (G+/I)

Home > Storage accounts Directory predefined

+ Create Restore Manage view Refresh Export to CSV Open query Assign tags Delete

Filter for any field... Subscription equals all Resource group equals all Location equals all Add filter

Showing 0 to 0 of 0 records. No grouping List view

Name ↑↓	Type ↑↓	Kind ↑↓	Resource group ↑↓	Location ↑↓	Subscription ↑↓
No storage accounts to display Create a storage account to store up to 500TB of data in the cloud. Use a general-purpose storage account to store object data, use a NoSQL data store, define and use queues for message processing, and set up file shares in the cloud. Use the Blob storage account and the hot or cool access tiers to optimize your costs based on how frequently your object data is accessed. Learn more					

[Give feedback](#)

Click on + Create and fill the fields as in the following figure.

Microsoft Azure (Preview) Upgrade Search resources, services, and docs (G+/I)

Home > Storage accounts > Create a storage account

Basics Advanced Networking Data protection Encryption Tags Review + create

Azure Storage is a Microsoft-managed service providing cloud storage that is highly available, secure, durable, scalable, and redundant. Azure Storage includes Azure Blobs (objects), Azure Data Lake Storage Gen2, Azure Files, Azure Queues, and Azure Tables. The cost of your storage account depends on the usage and the options you choose below. [Learn more about Azure storage accounts](#)

Project details

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription * Azure subscription 1

Resource group * ExpenseGroup_Demo [Create new](#)

Instance details

Storage account name * storagedemoexpense

Region * (Europe) West Europe [Deploy to an edge zone](#)

Performance * ☒ Standard: Recommended for most scenarios (general-purpose v2 account) ☐ Premium: Recommended for scenarios that require low latency.

Redundancy * Geo-redundant storage (GRS) ☒ Make read access to data available in the event of regional unavailability.

[Previous](#) [Next](#) [Review + create](#)

[Give feedback](#)

Leave all the options in the other tabs as they are and then click on Create.

Microsoft Azure (Preview) Upgrade Search resources, services, and docs (G+/I)

Home > **storagedemoexpense** Storage account

Search

Upload Open in Explorer Delete Move Refresh Open in mobile Feedback

Overview

- Activity log
- Tags
- Diagnose and solve problems
- Access Control (IAM)
- Data migration
- Events
- Storage browser
- Storage Mover

Data storage

- Containers
- File shares
- Queues
- Tables

Security + networking

- Networking
- Front Door and CDN
- Access keys
- Shared access signature
- Encryption
- Microsoft Defender for Cloud

Data management

- Redundancy

Essentials View Cost JSON View

Resource group (move) : ExpenseGroup_Demo Performance : Standard
 Location : westeurope Replication : Read-access geo-redundant storage (RA-GRS)
 Primary/Secondary Location : Primary: West Europe, Secondary: North Europe Account kind : StorageV2 (general purpose v2)
 Subscription (move) : Azure subscription_1 Provisioning state : Succeeded
 Subscription ID : 2305b9dc-c718-4afd-bcdf-21cf897a7188 Created : 11/1/2024, 16:48:38
 Disk state : Primary: Available, Secondary: Available

Tags (edit) : Add tags

Properties Monitoring Capabilities (7) Recommendations (0) Tutorials Tools + SDKs

Blob service

- Hierarchical namespace : Disabled
- Default access tier : Hot
- Blob anonymous access : Disabled
- Blob soft delete : Enabled (7 days)
- Container soft delete : Enabled (7 days)
- Versioning : Disabled
- Change feed : Disabled
- NFS v3 : Disabled
- Allow cross-tenant replication : Disabled

File service

- Large file share : Disabled
- Identity-based access : Not configured
- Default share-level permissions : Disabled
- Soft delete : Enabled (7 days)

Security

- Require secure transfer for REST API operations : Enabled
- Storage account key access : Enabled
- Minimum TLS version : Version 1.2
- Infrastructure encryption : Disabled

Networking

- Allow access from : All networks
- Number of private endpoint connections : 0
- Network routing : Microsoft network routing
- Access for trusted Microsoft services : Yes

In the left panel navigate to Access keys.

Microsoft Azure (Preview) Upgrade Search resources, services, and docs (G+/I)

Home > storagedemoexpense

storagedemoexpense | Access keys ☆ ...

Search

Set rotation reminder Refresh Give feedback

Access keys authenticate your applications' requests to this storage account. Keep your keys in a secure location like Azure Key Vault, and replace them often with new keys. The two keys allow you to replace one while still using the other.

Remember to update the keys with any Azure resources and apps that use this storage account.
 Learn more about managing storage account access keys

Storage account name : storagedemoexpense

key1 Rotate key
 Last rotated: 11/1/2024 (0 days ago)

Key : oqGGx00OrARjZ7BkderRfYyK2LjHhGWQfH8tUbpVtkZFMv8jg8ShDp439KsBD... Hide

Connection string : DefaultEndpointsProtocol=https;AccountName=storagedemoexpense;AccountKe... Hide

key2 Rotate key
 Last rotated: 11/1/2024 (0 days ago)

Key : bx1myBt5HjgQJ4Ejfbt14Pu4jDKmfZSb7NZfM5TjpeEgEL2+wk0wKQlnfH0xqYo3... Hide

Connection string : DefaultEndpointsProtocol=https;AccountName=storagedemoexpense;AccountKe... Hide

Consider, for example, **key1**. Copy the related connection string.

Open your function app in a new tab and navigate to Configuration.

The screenshot shows the Microsoft Azure portal interface. The top navigation bar includes the Microsoft Azure logo, an 'Upgrade' button, a search bar, and a user profile. The left sidebar contains a navigation menu with categories like Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Microsoft Defender for Cloud, Events (preview), Functions, App keys, App files, Proxies, Deployment slots, Deployment Center, and Settings. The 'Settings' section is expanded, showing 'Configuration' as the selected option. The main content area displays the 'Configuration' page for the 'FunctionExpenseDemo' Function App. It has three tabs: 'Application settings', 'Function runtime settings', and 'General settings'. The 'Application settings' tab is active, showing a list of application settings. The settings are organized into two sections: 'Application settings' and 'Connection strings'. The 'Application settings' section has a '+ New application setting' button and a 'Filter application settings' input. Below this is a table with columns: Name, Value, Source, Deployment slot setting, Delete, and Edit. The table lists four settings: APPLICATIONINSIGHTS_CONNECTION_STRING, AzureWebJobsSecretStorageType, FUNCTIONS_EXTENSION_VERSION, and FUNCTIONS_WORKER_RUNTIME. The 'Connection strings' section has a '+ New connection string' button and a 'Filter connection strings' input. Below this is a table with columns: Name, Value, Source, Type, Deploy..., Delete, and Edit. The table is currently empty, with a message '(no connection strings to display)'.

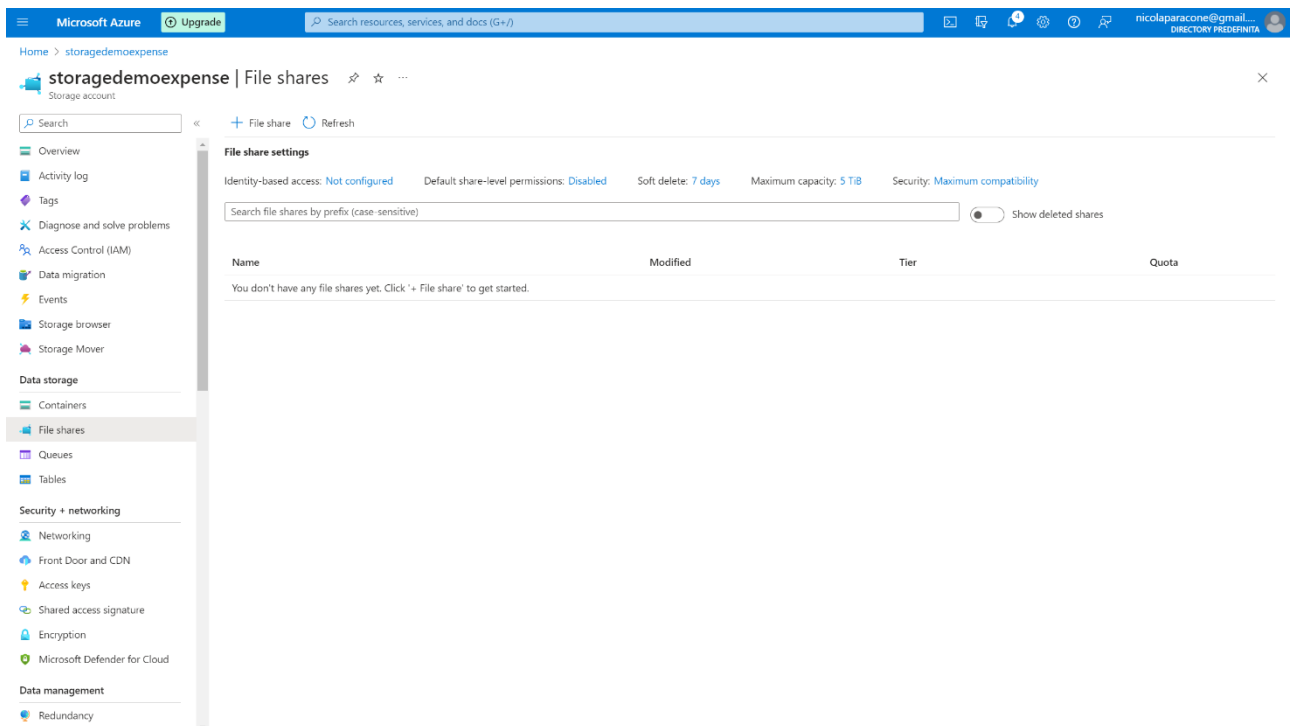
Click on + New application setting. As Name of the new application setting give AzureWebJobsStorage, and as Value paste the connection string. Click Ok.

The screenshot shows the Microsoft Azure portal interface, similar to the previous one, but with a dialog box open. The dialog box is titled 'Add/Edit application setting' and has a close button (X) in the top right corner. It contains two input fields: 'Name' and 'Value'. The 'Name' field is filled with 'AzureWebJobsStorage'. The 'Value' field is filled with 'DefaultEndpointsProtocol=https;AccountName=storageaccountdemoexpense;AccountKey=oqGgs00OraRjZ78kd...'. There is a checkbox labeled 'Deployment slot setting' which is currently unchecked. At the bottom of the dialog box are 'Ok' and 'Cancel' buttons. The background shows the 'Configuration' page of the 'FunctionExpenseDemo' Function App, with the 'Application settings' tab selected.

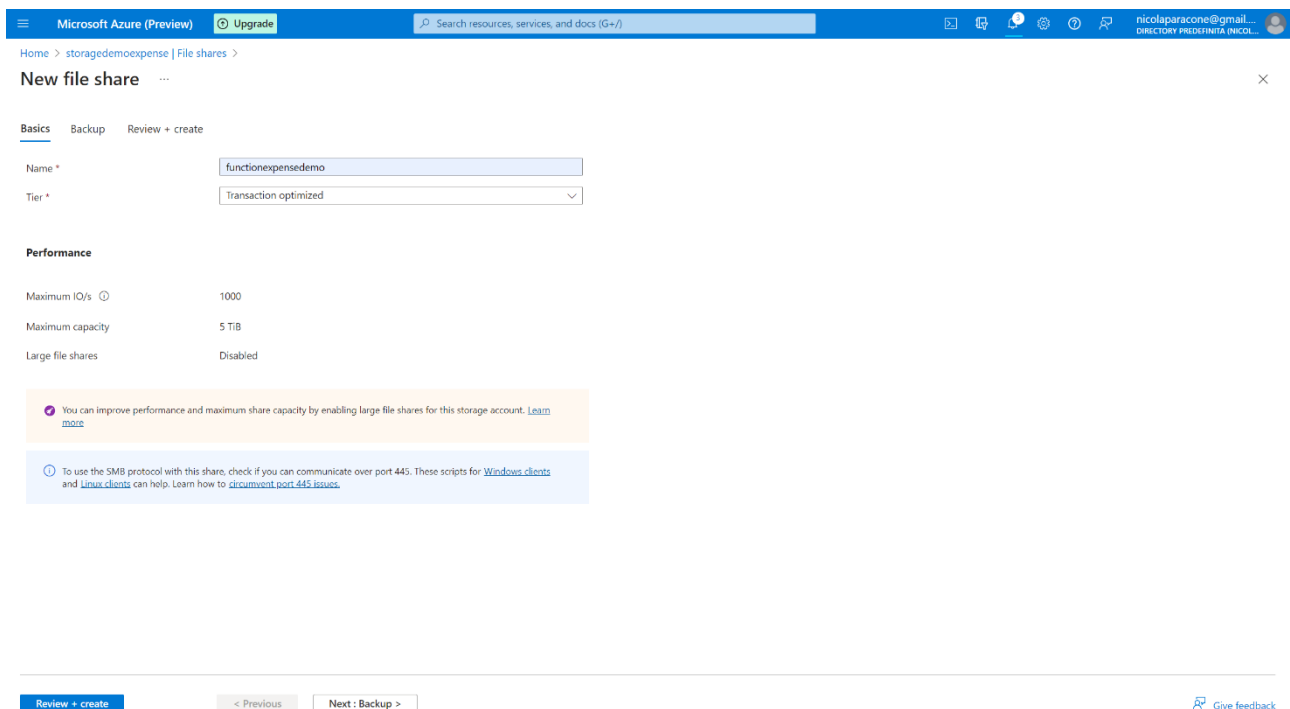
AzureWebJobsStorage is used to specify the storage account where the function's metadata and trigger-related information are stored.

Using the same connection string, create another application setting called WEBSITE_CONTENTAZUREFILECONNECTIONSTRING.

Next, navigate to File shares in the left panel inside storagedemoexpense.

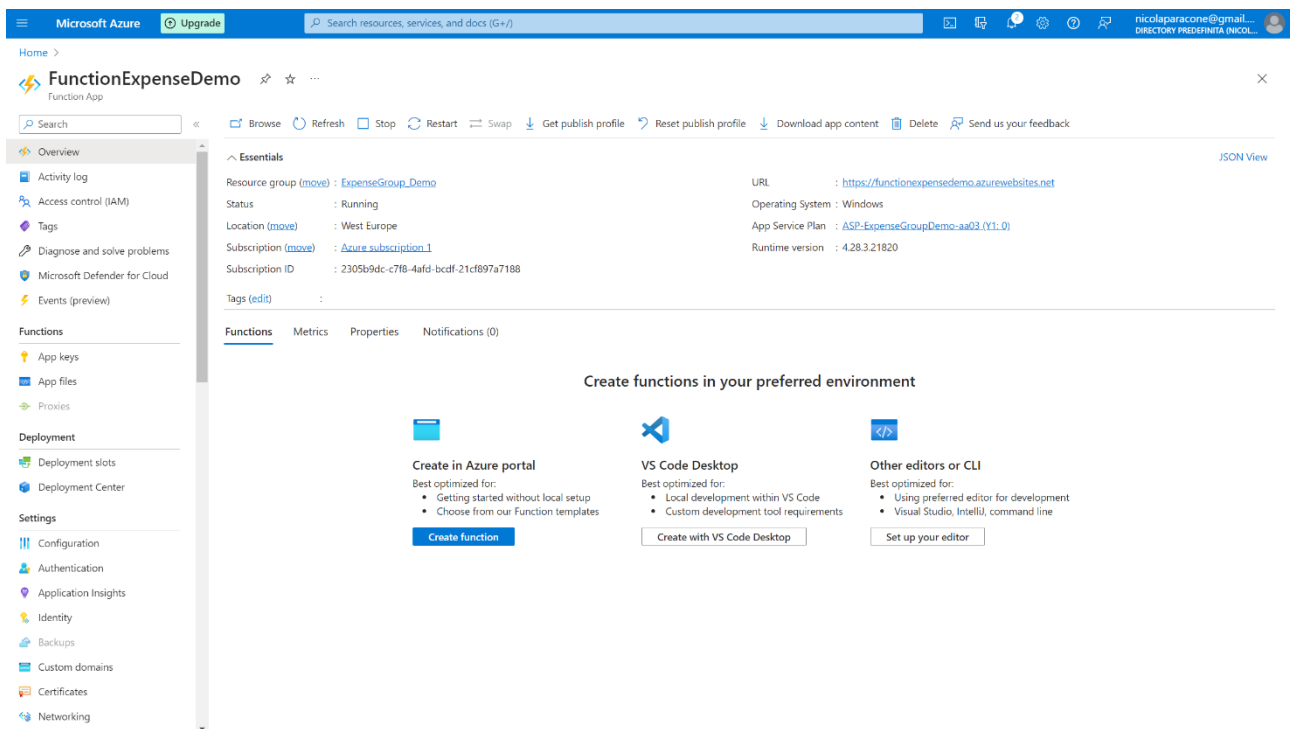


Click on + File share and create a new file share as in the following figure.



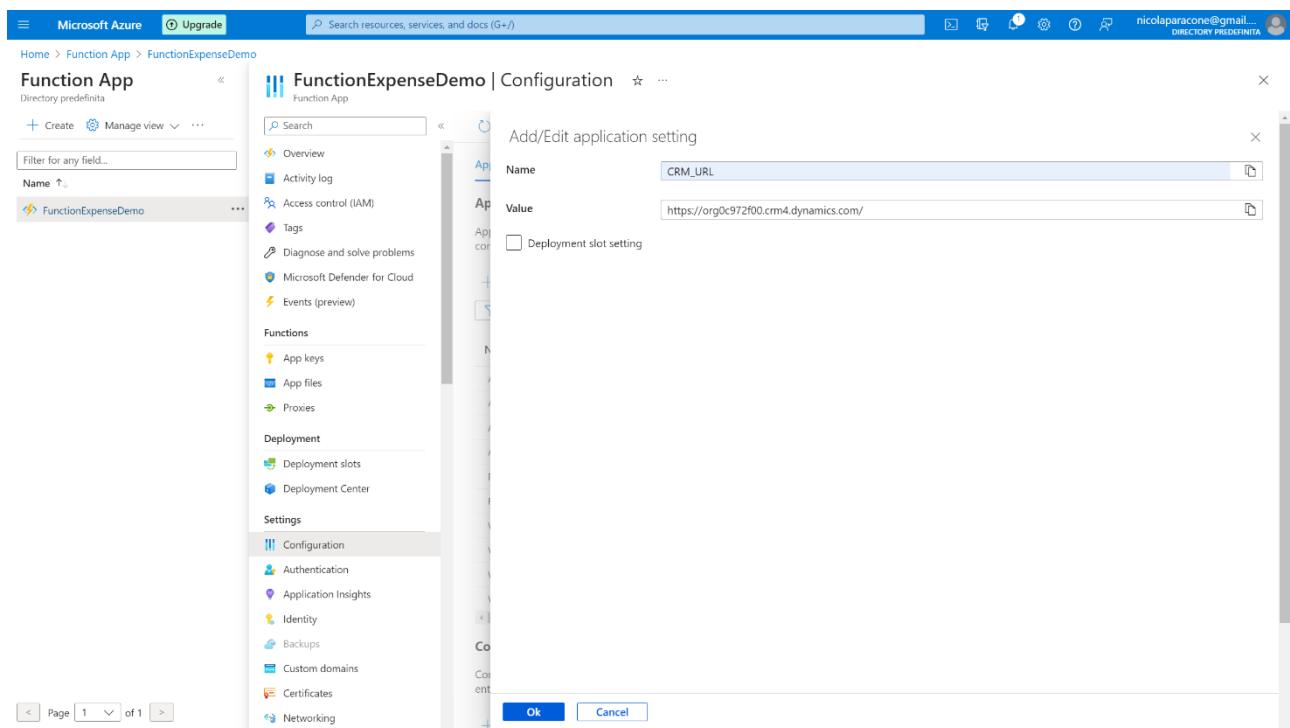
Now, go back to the configuration windows of the function app and create a new application setting with name WEBSITE_CONTENTSHARE and value equal to functionexpensedemo. Save the new configuration and navigate to Overview.

Notice that the warning will have disappeared.



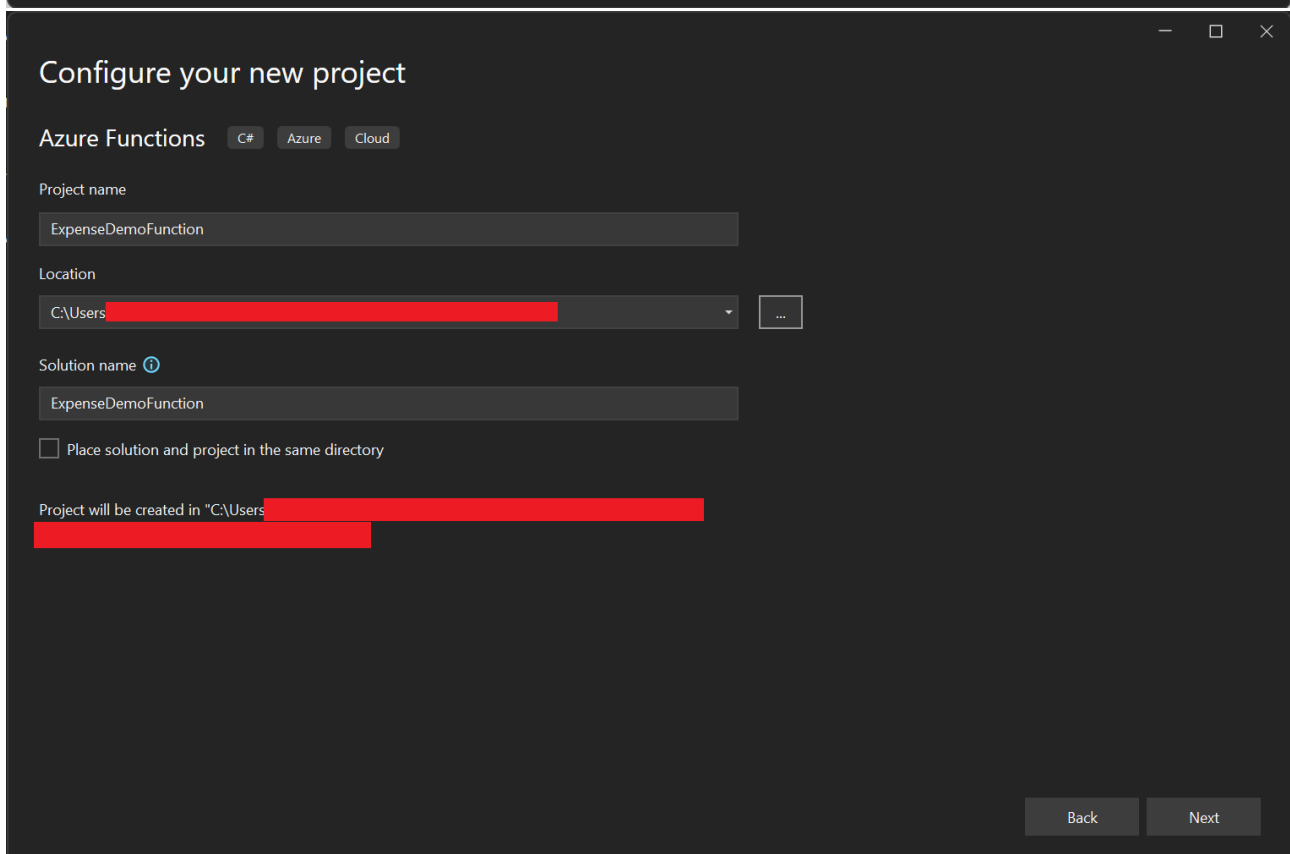
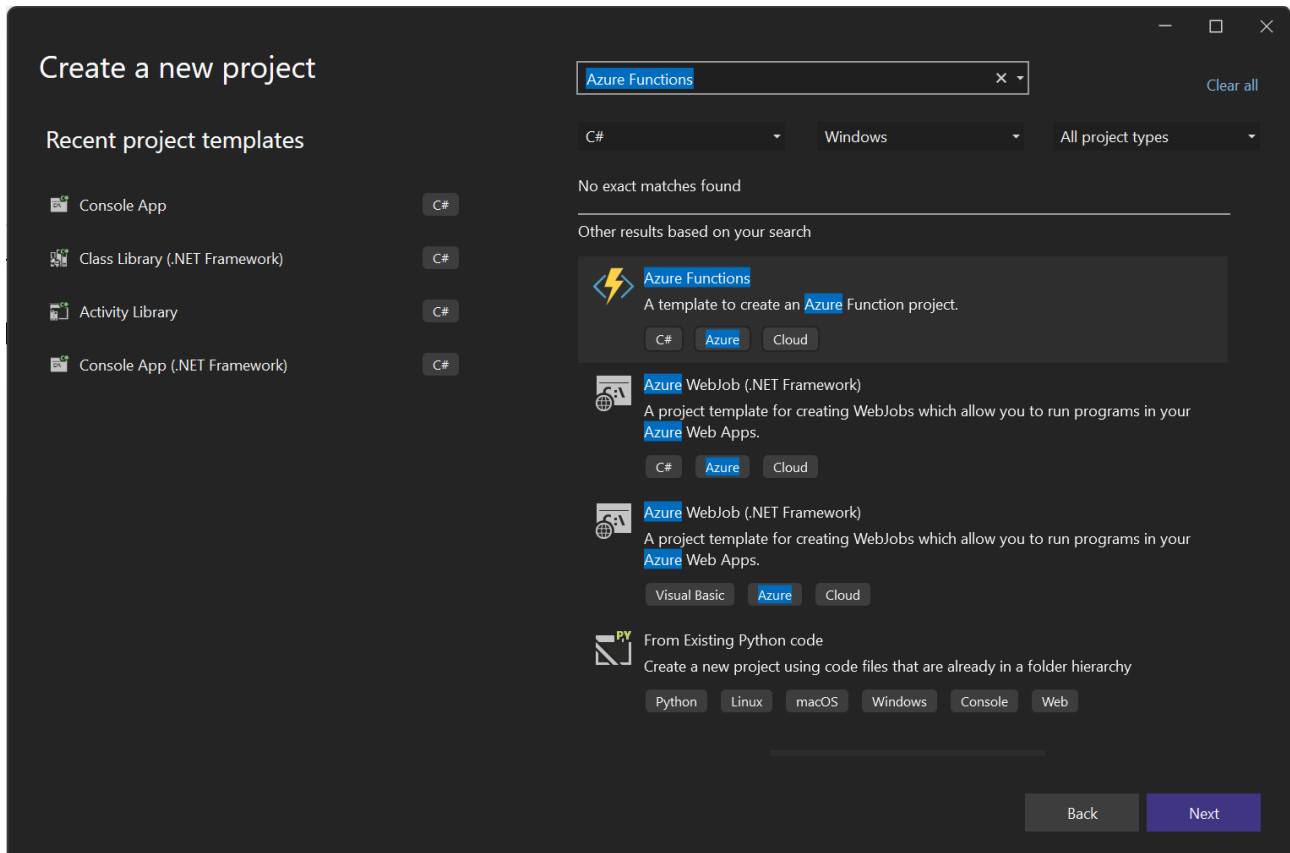
This entire procedure is completely avoided if you have a proper Azure subscription, since during the creation of the function app the storage account tab will appear.

As a final step, we create an application string called CRM_URL with value equal to the URL of our Dynamics 365 environment.



3. Writing an Azure function in Visual Studio

Let us create a new project in Visual Studio of type Azure Functions.



Here we have the possibility of choosing among many types of Azure functions.

Additional information

Azure Functions C# Azure Cloud

Functions worker ⓘ

.NET 8.0 Isolated (Long Term Support)

Function ⓘ

Http trigger

Dapr Service Invocation Trigger

Dapr Topic Trigger

Durable Functions Orchestration

Event Grid trigger

Event Hub trigger

Http trigger

Kusto input binding isolated

Kusto output binding isolated

Queue trigger

RabbitMQ trigger

Service Bus Queue trigger

Service Bus Topic trigger

SignalR

SQL input binding isolated

SQL output binding isolated

SQL trigger

Timer trigger

Back Create

To begin, we start with an HTTP triggered function. It operates by responding to HTTP requests. In this model, when an HTTP request is made to a specific endpoint associated with the function, the function is triggered and executes the code it contains. The function may generate a response, such as HTML content or JSON data, which is then sent back to the client that initiated the HTTP request. This approach is commonly used for building APIs, handling webhooks, and performing various tasks associated with HTTP events in a scalable and efficient manner.

Additional information

Azure Functions C# Azure Cloud

Functions worker ⓘ

.NET 8.0 Isolated (Long Term Support)

Function ⓘ

Http trigger

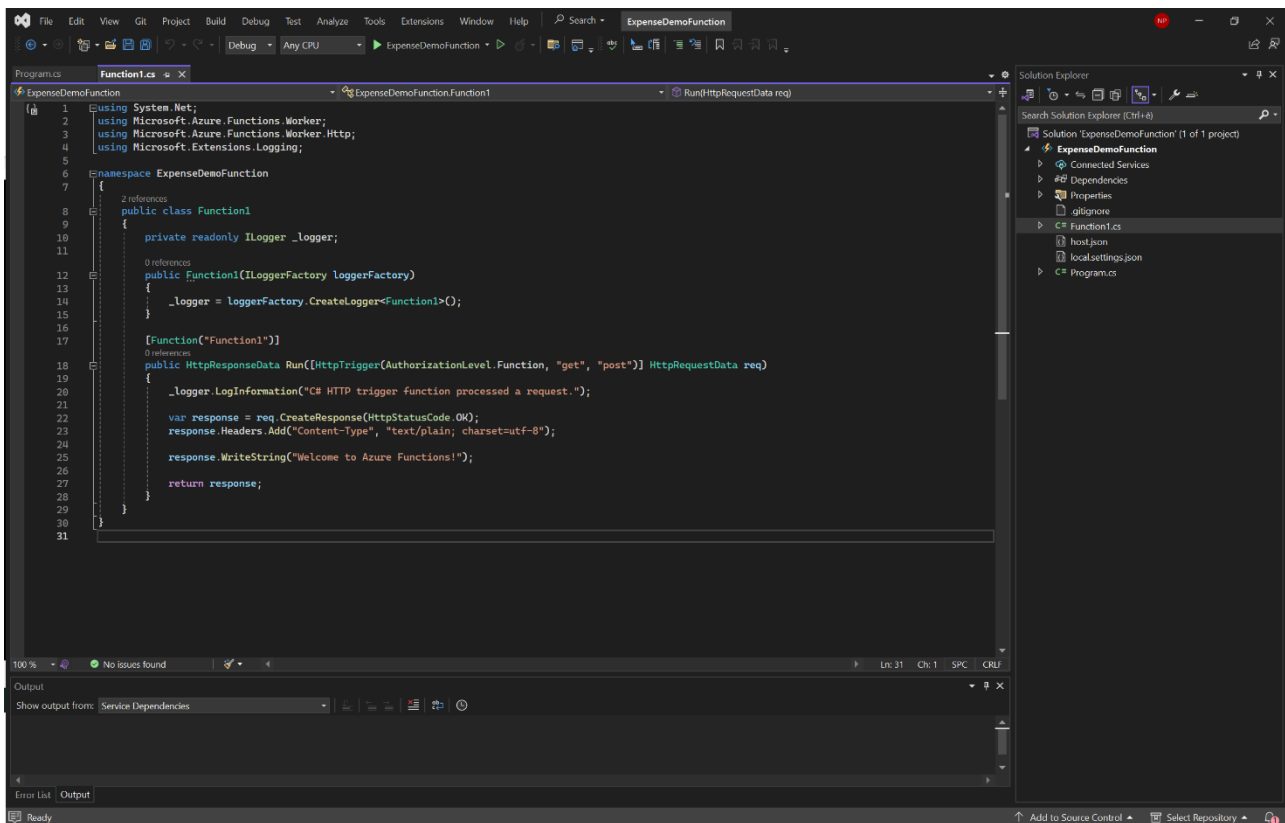
☒ Use Azurite for runtime storage account (AzureWebJobsStorage) ⓘ

☐ Enable Docker ⓘ

Authorization level ⓘ

Function

Back Create



This is a basic Azure Functions HTTP-triggered function in C#. This function responds to both HTTP GET and POST requests with a simple "Welcome to Azure Functions!" message. Here's a breakdown of the key components in the code:

1. **Logger:**

- We have a logger injected into our function via the constructor. This logger is an instance of `ILogger<Function1>`. It allows us to log information, warnings, errors, etc., to the Azure Functions logging system.

2. **Function Attribute:**

- The `[Function]` attribute is used to define our function. In this case, the function is named `Function1`.

3. **Trigger Attribute:**

- The `[HttpTrigger]` attribute is applied to the `Run` method, specifying that this function should be triggered by HTTP requests.
- `AuthorizationLevel.Function` indicates that the function is secured using a function-specific key.
- The allowed HTTP methods are GET and POST.

4. Run Method:

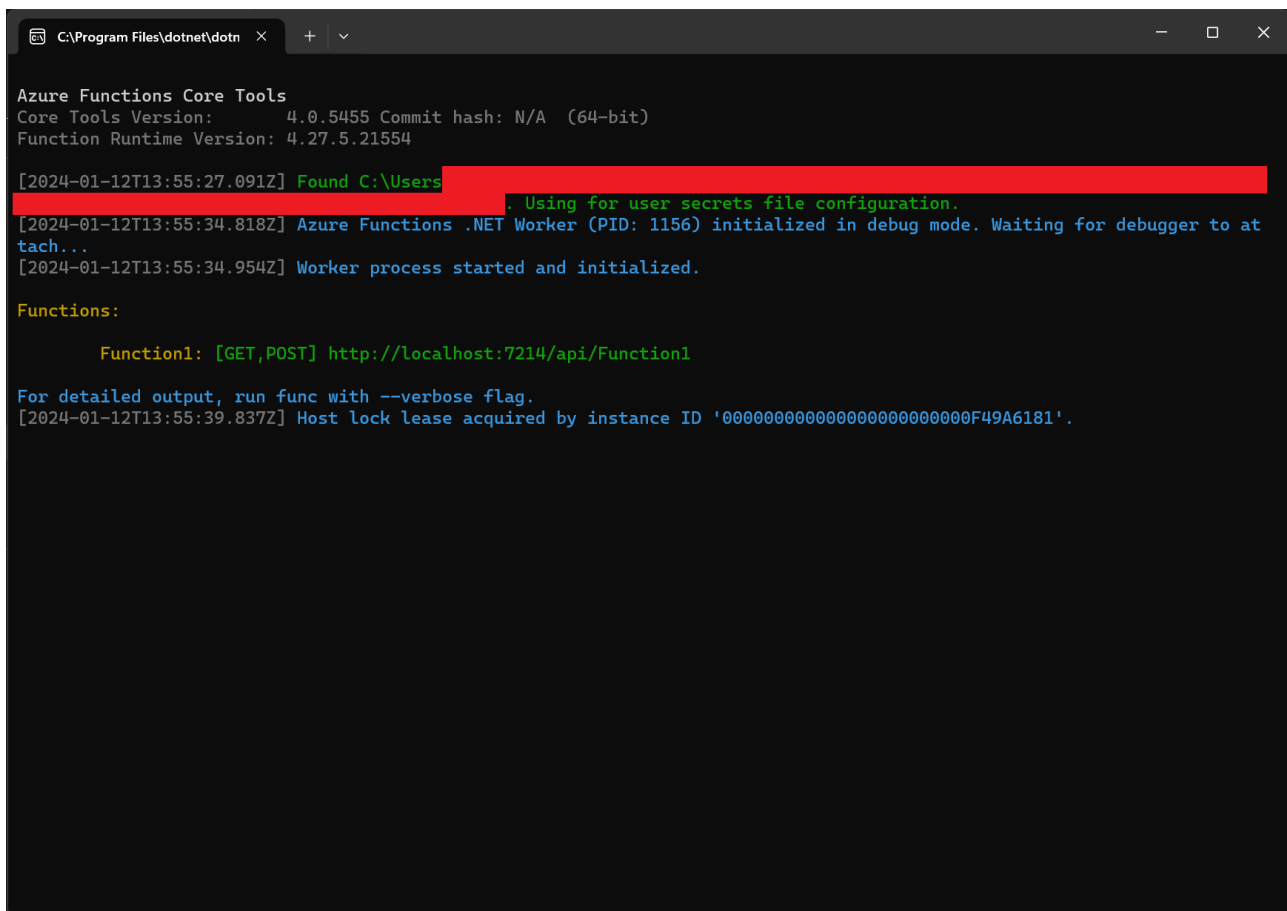
- The Run method is the entry point for our function. It receives an `HttpRequestData` object representing the incoming HTTP request.
- The function logs an informational message using the injected logger.
- It then creates an `HttpResponseData` object to construct the HTTP response.
- The response is set to have an OK status (200) and a "text/plain" content type.
- The response body is set to "Welcome to Azure Functions!"

5. Return Statement:

- The function returns the constructed `HttpResponseData` object, which will be sent back as the HTTP response.

This is a simple example, but we can expand on it by adding logic to process the incoming request, accessing query parameters, handling request headers, and performing more complex operations based on the client's request.

Let us run this function from our local machine.



```
C:\Program Files\dotnet\dotn x + v

Azure Functions Core Tools
Core Tools Version: 4.0.5455 Commit hash: N/A (64-bit)
Function Runtime Version: 4.27.5.21554

[2024-01-12T13:55:27.091Z] Found C:\Users\ [REDACTED]
[2024-01-12T13:55:34.818Z] Azure Functions .NET Worker (PID: 1156) initialized in debug mode. Waiting for debugger to attach...
[2024-01-12T13:55:34.954Z] Worker process started and initialized.

Functions:

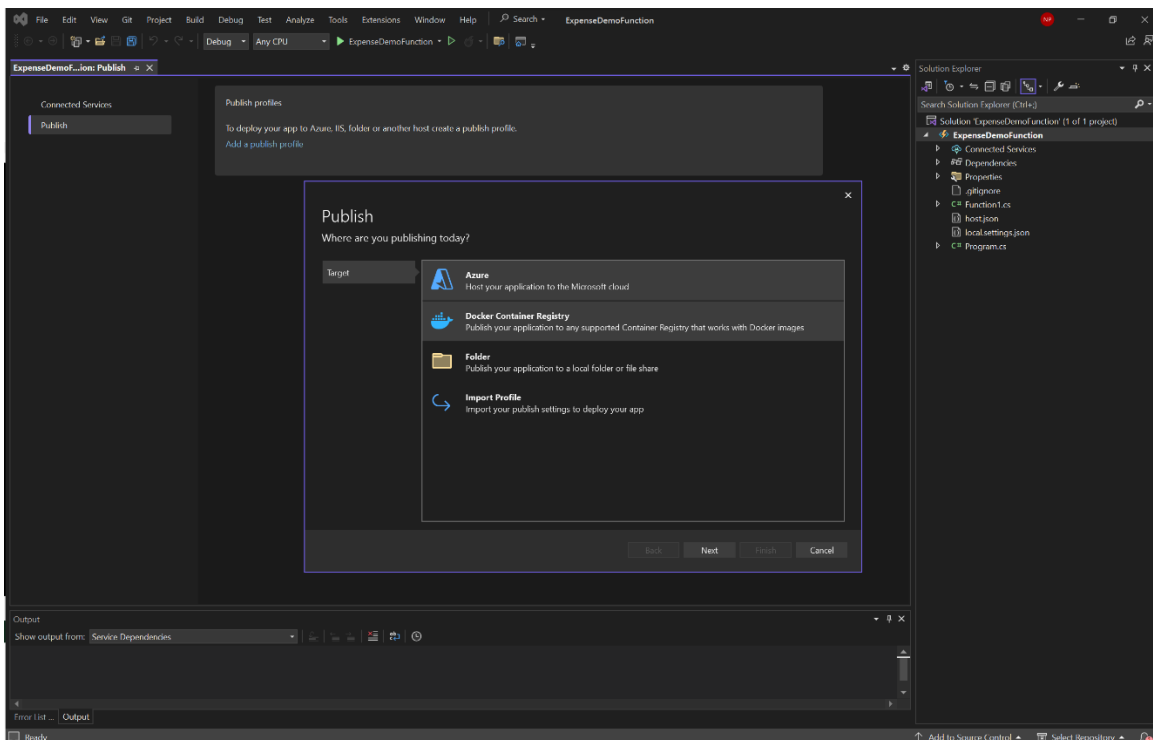
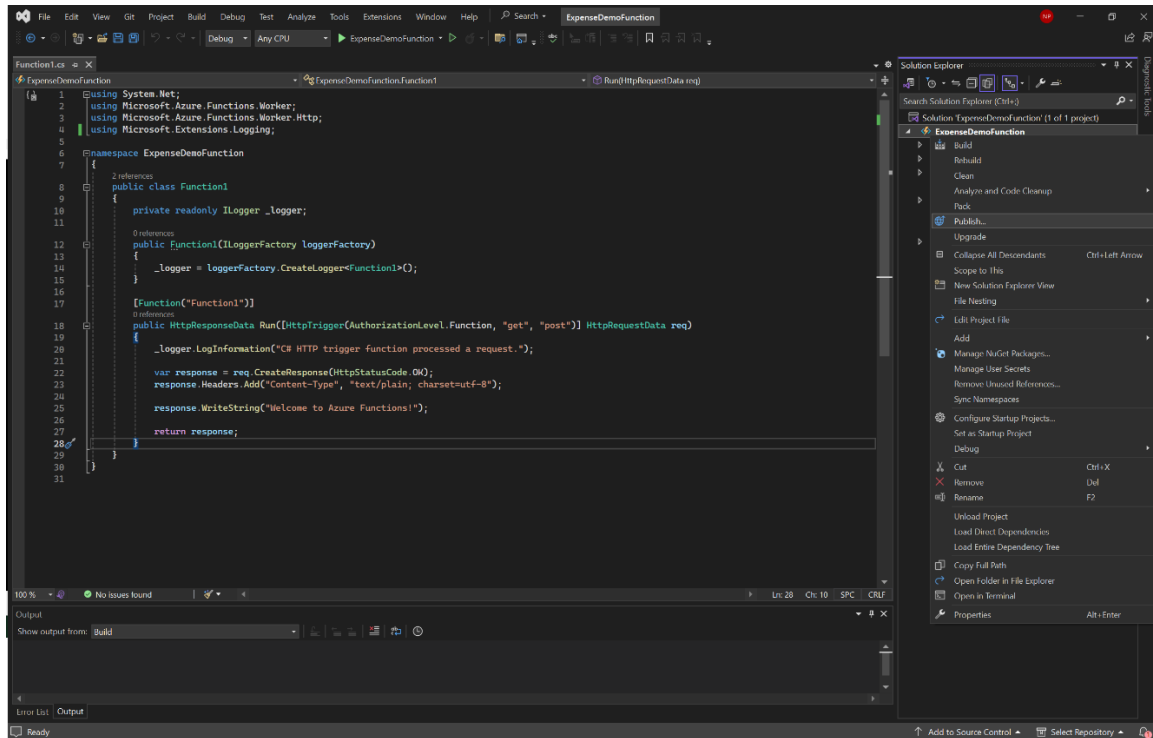
    Function1: [GET,POST] http://localhost:7214/api/Function1

For detailed output, run func with --verbose flag.
[2024-01-12T13:55:39.837Z] Host lock lease acquired by instance ID '000000000000000000000000F49A6181'.
```

The URL <http://localhost:7214/api/Function1> is a local development endpoint for our Azure Function with an HTTP trigger. If we paste it into a browser we will get the "Welcome to Azure Functions!" message on screen.

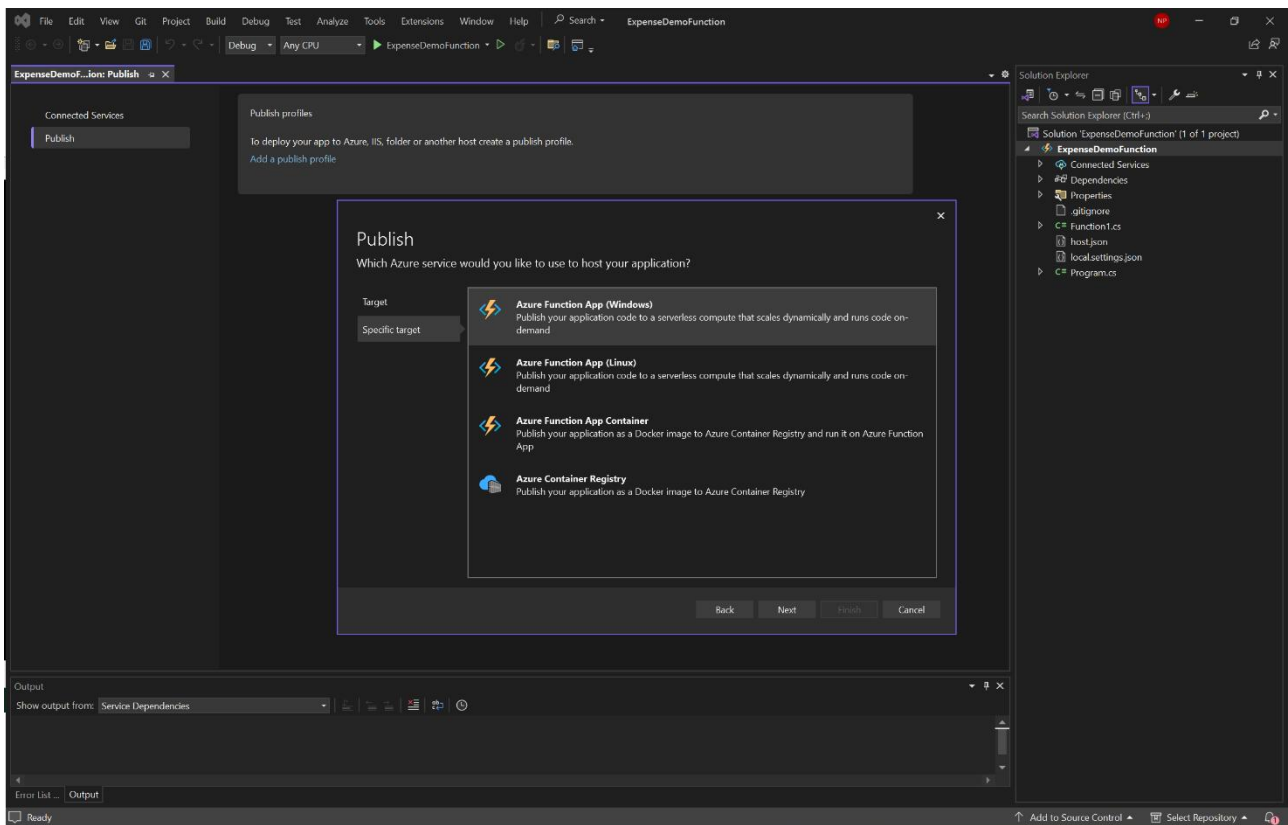
4. Publishing the function

Assuming that the function we have just created already contains the functionality we wanted, we can proceed to publish it. Click on Publish... as in the following figure.

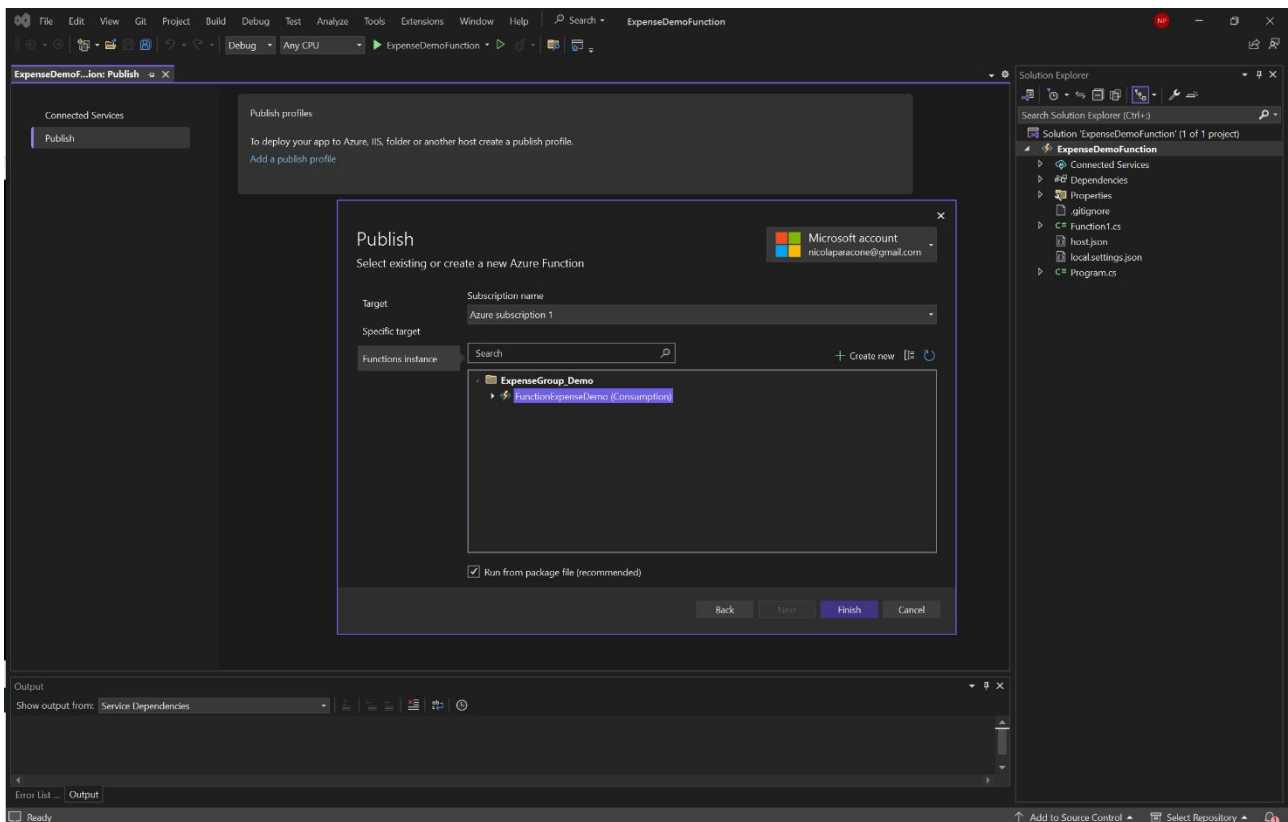


Choose Azure.

Choose Azure Function App (Windows).

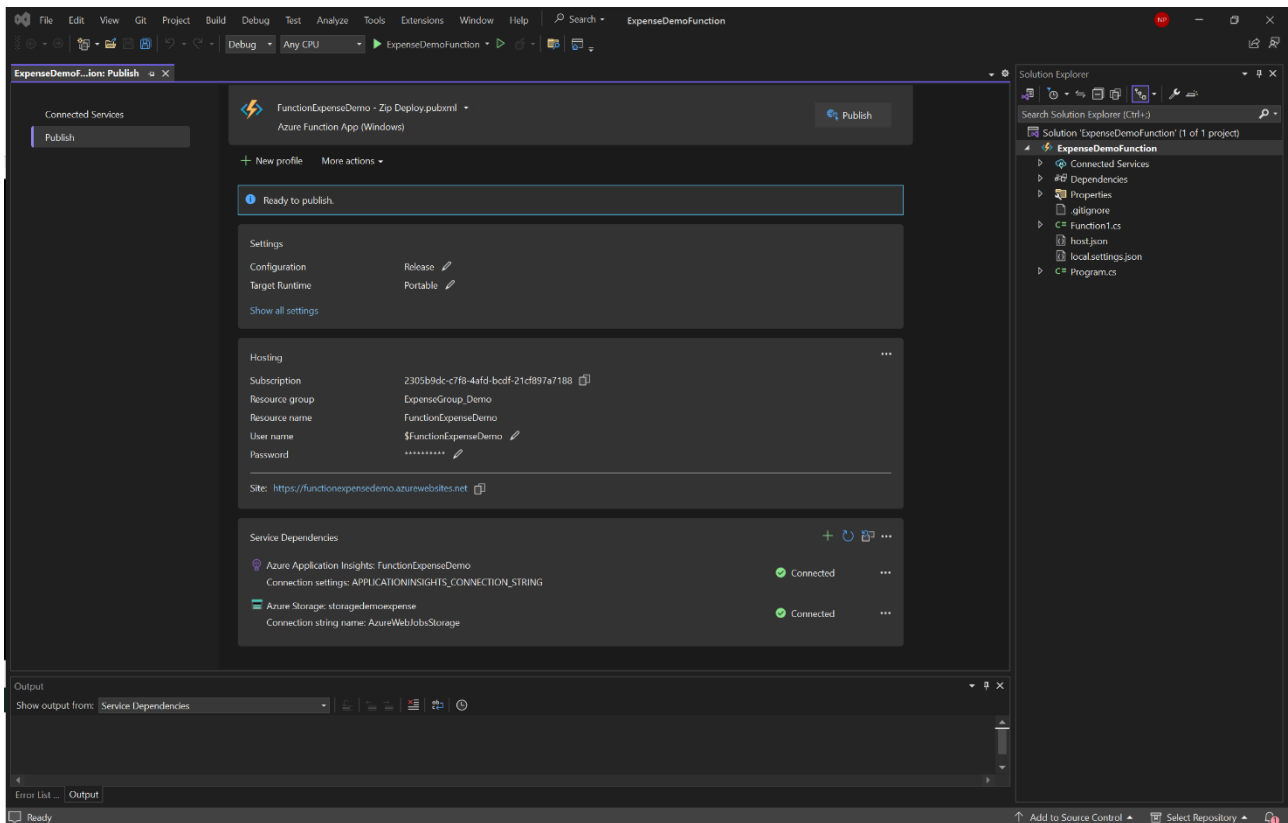


Choose the FunctionExpenseDemo function app we have previously created.

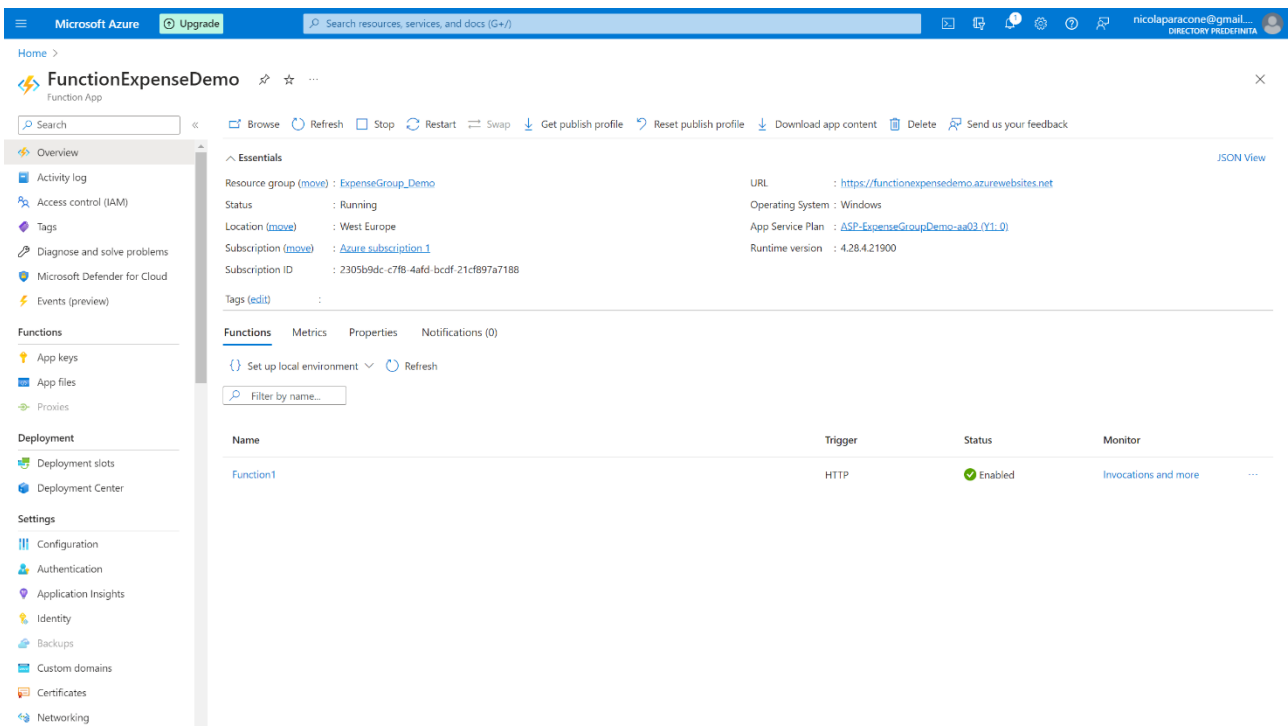


Then, click on Finish.

Once the following windows appears, click on .



Once the publishing operation is complete, we can navigate to our function app in Azure.



Here we will find our HTTP-triggered function.

Click on it and then click on Get Function Url.

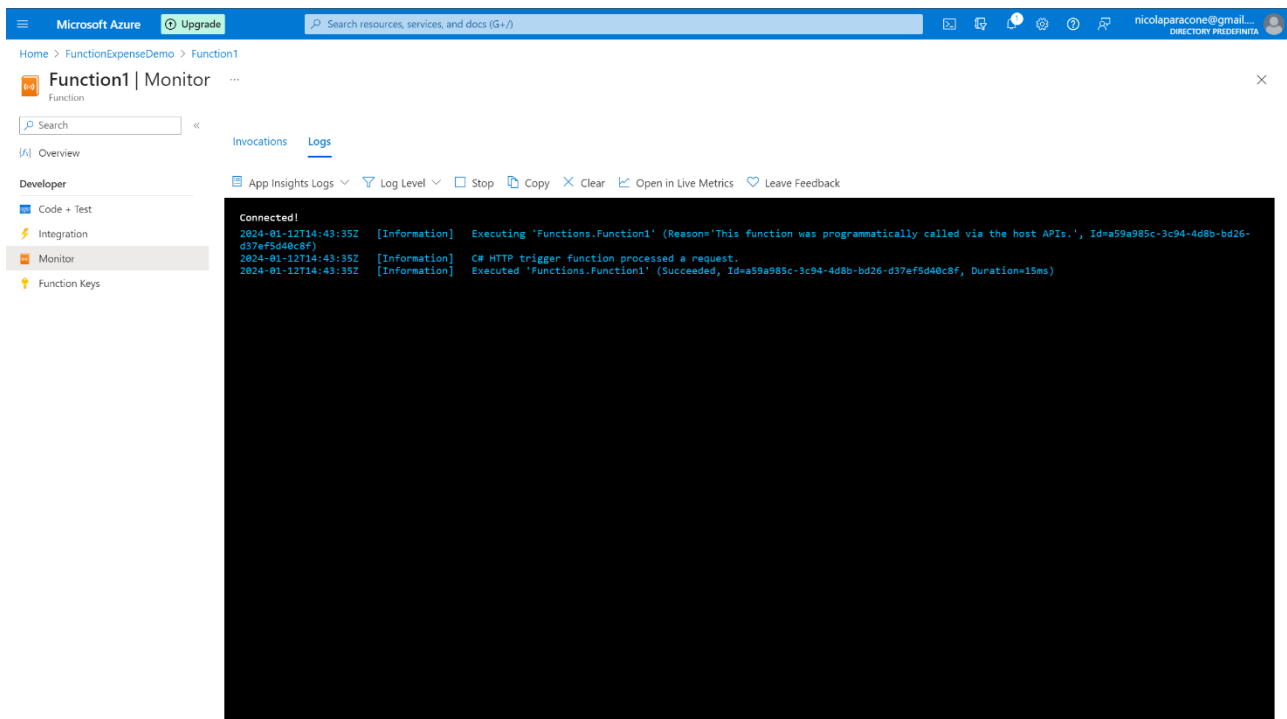
The screenshot shows the Microsoft Azure portal interface. At the top, the navigation bar includes 'Microsoft Azure', an 'Upgrade' button, a search bar, and a user profile. The main content area displays the 'Function1' resource page. A 'Get Function Url' dialog box is open, showing the URL 'https://functionexpensedemo.azurewebsites.net/api/Function1?code=nSliKWPqOhpBN3Dxb1Ap6EoCxe...' and an 'OK' button. Below the dialog, the 'Overview' tab is selected, showing subscription details and two empty line charts for 'Total Execution Count' and 'Successful Execution Count'.

Copy the URL. At this point, open Postman and paste that URL in a new request.

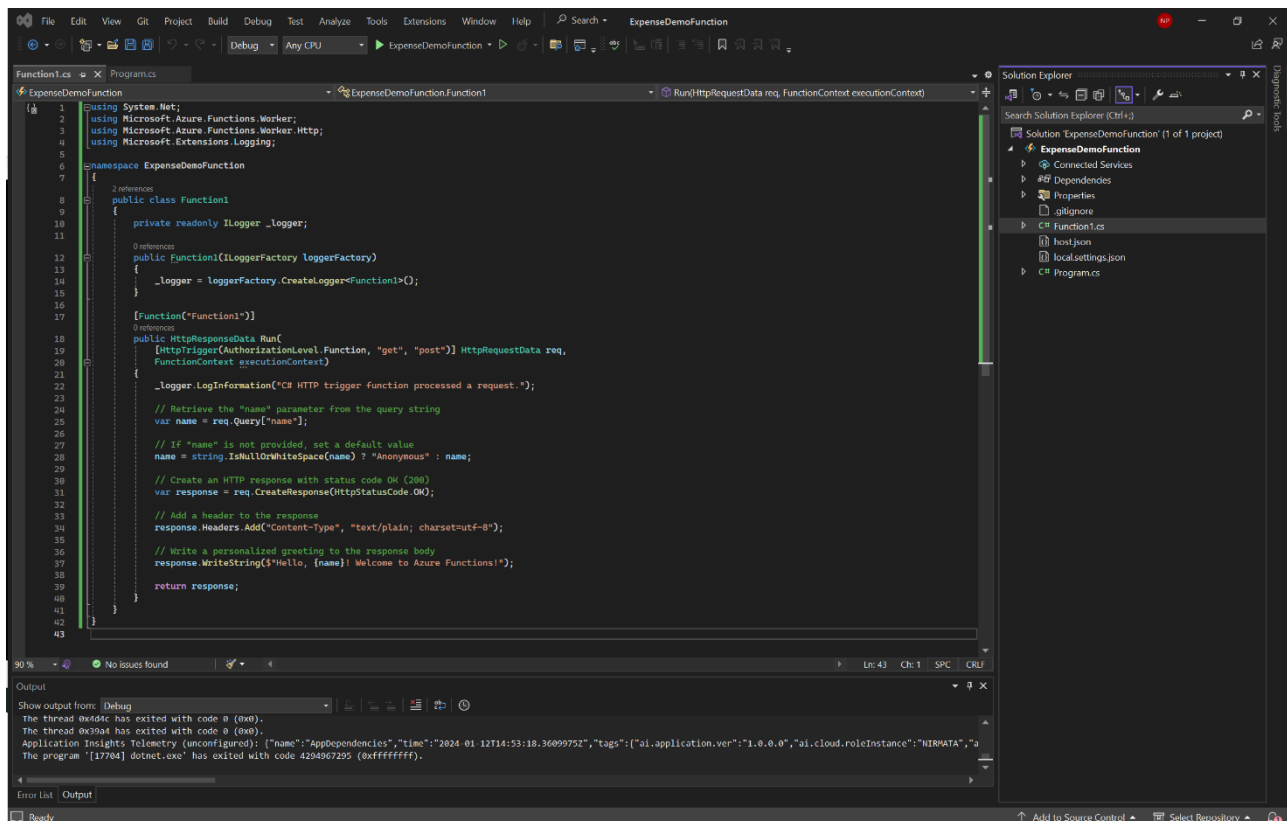
The screenshot shows the Postman application interface. A new POST request is created with the URL 'https://functionexpensedemo.azurewebsites.net/api/Function1?code=nSliKWPqOhpBN3Dxb1Ap6EoCxe...'. The 'Params' tab is selected, showing a query parameter 'code' with the value 'nSliKWPqOhpBN3Dxb1Ap6EoCxe...'. The 'Body' tab is also visible, showing a 'Welcome to Azure Functions!' message.

Notice that we will get again our "Welcome to Azure Functions!" message.

In the Monitor section we will find instead the Logs produced by our function.



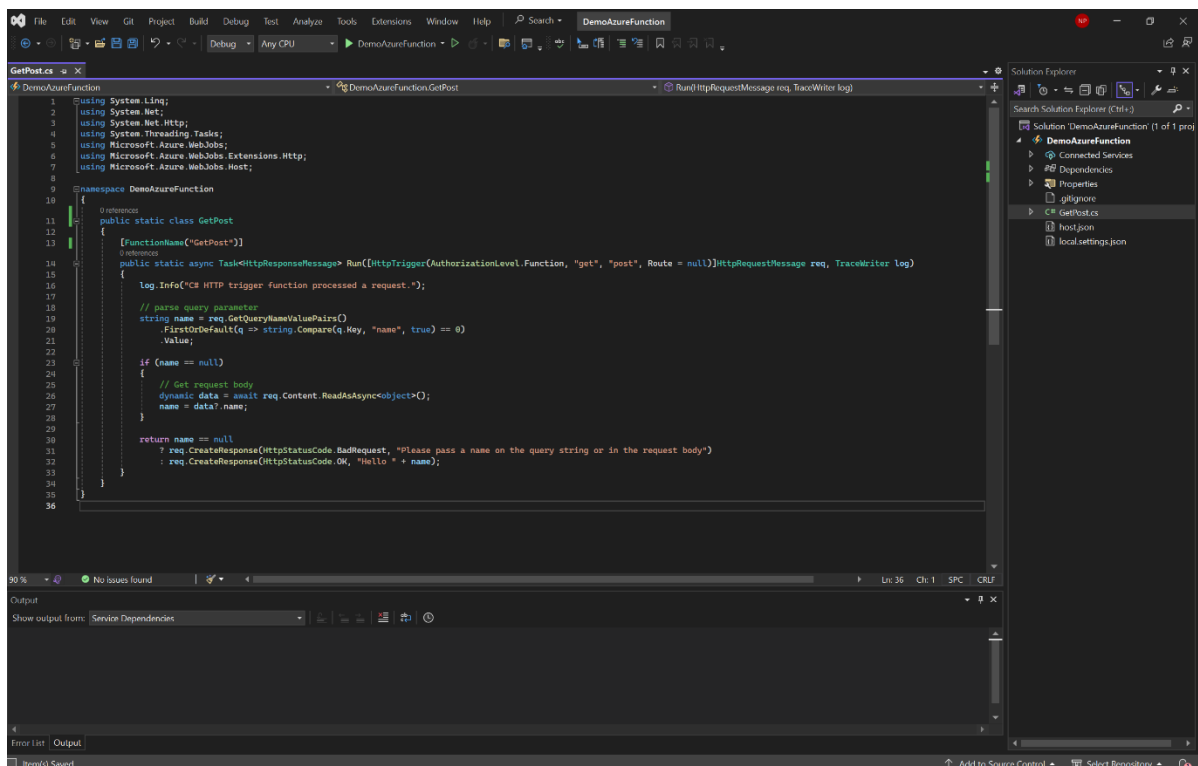
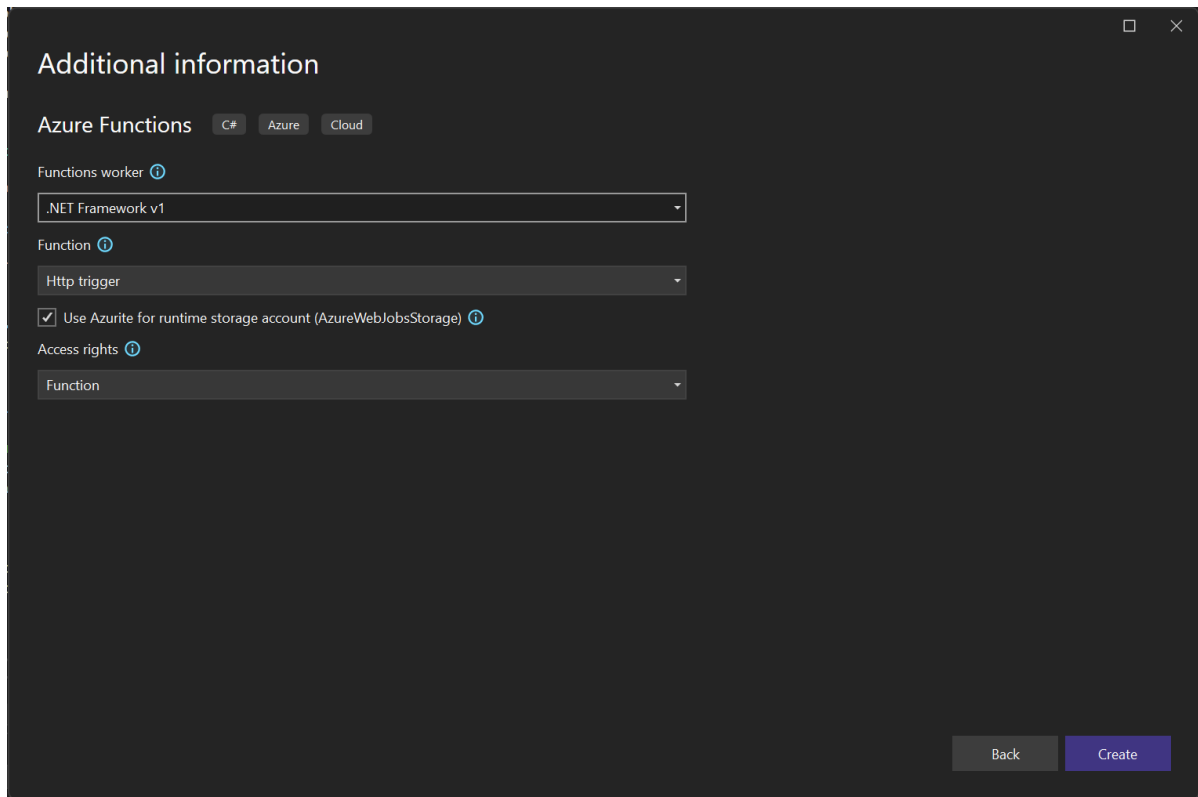
In the following code, the request also accepts a name parameter.



Assuming that the endpoint is <http://localhost:7214/api/Function1>, if we replace it with <http://localhost:7214/api/Function1?name=John> the message we will get is "Hello, John! Welcome to Azure Functions!".

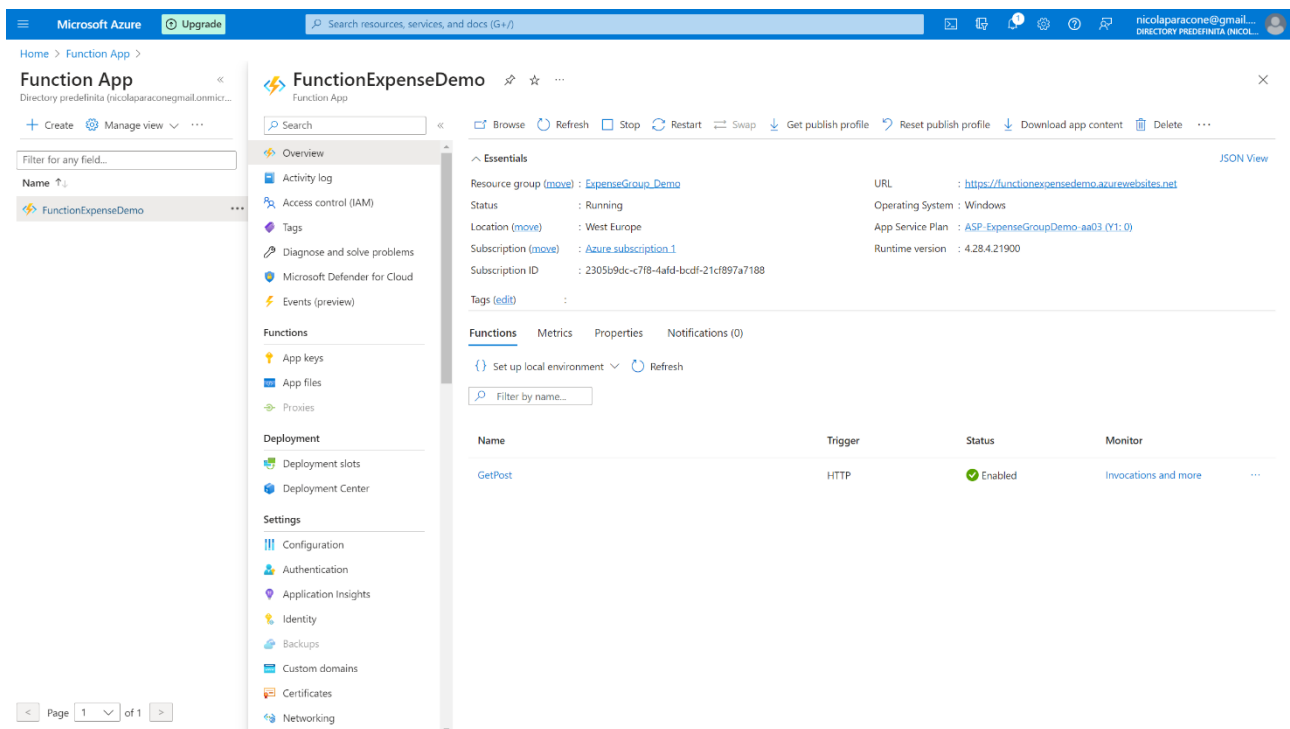
5. Another example using .NET Framework v1

Let us create another HTTP-triggered Azure function in Visual Studio. This time, however, we will choose .NET Framework v1 as Functions worker.

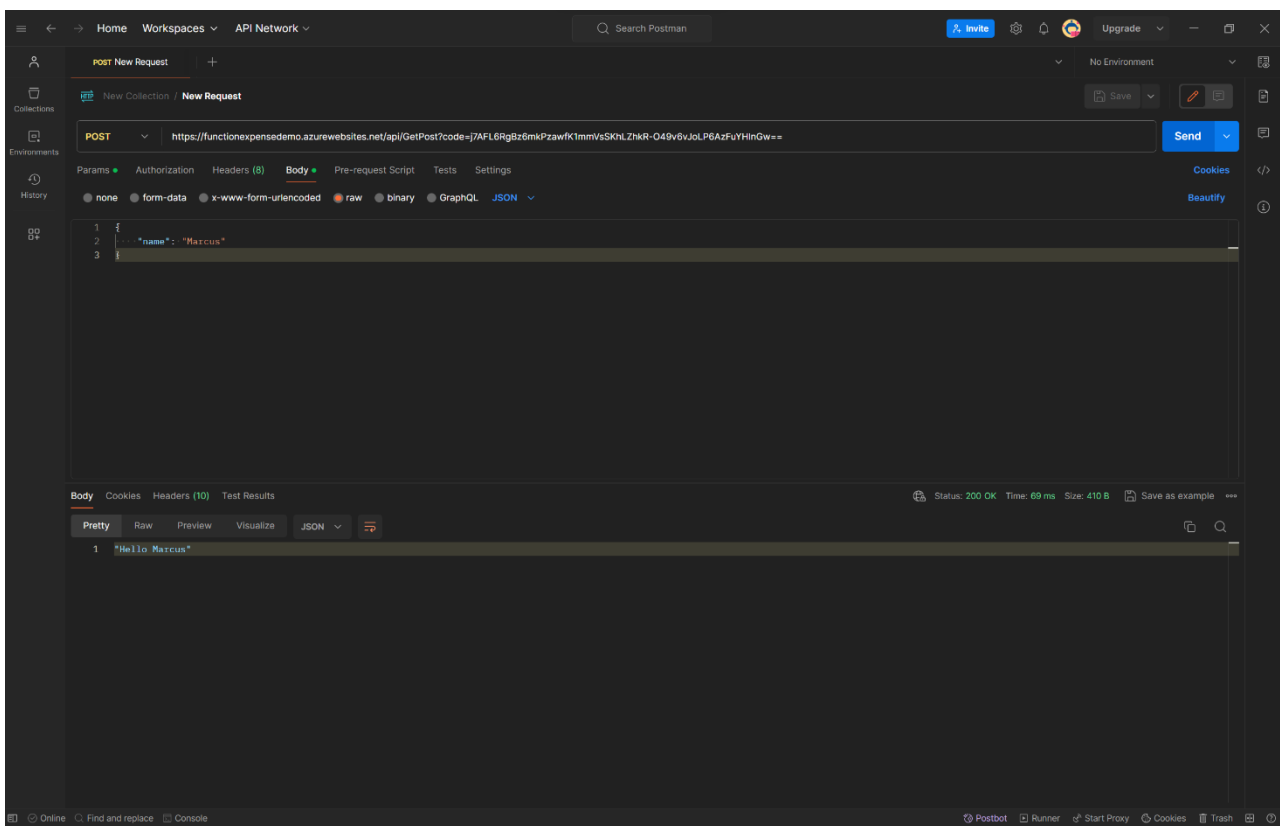


Notice that we changed the name of the function into GetPost.

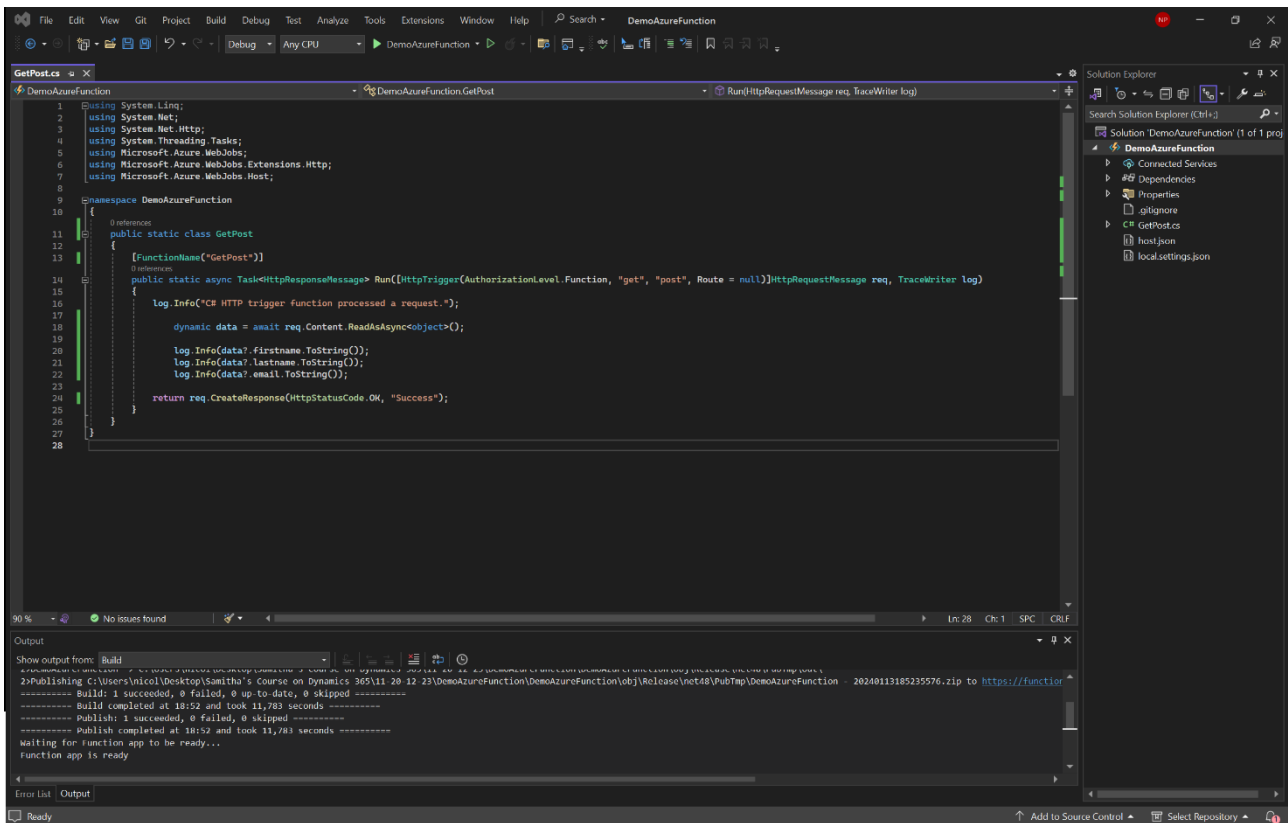
After building our function, we proceed to publish it in Azure.



Once we have acquired the function URL, we can trigger the function by sending a request from Postman. There we can provide a JSON containing the name value we want to be used in our function.



Suppose that we have changed our function into the following one.



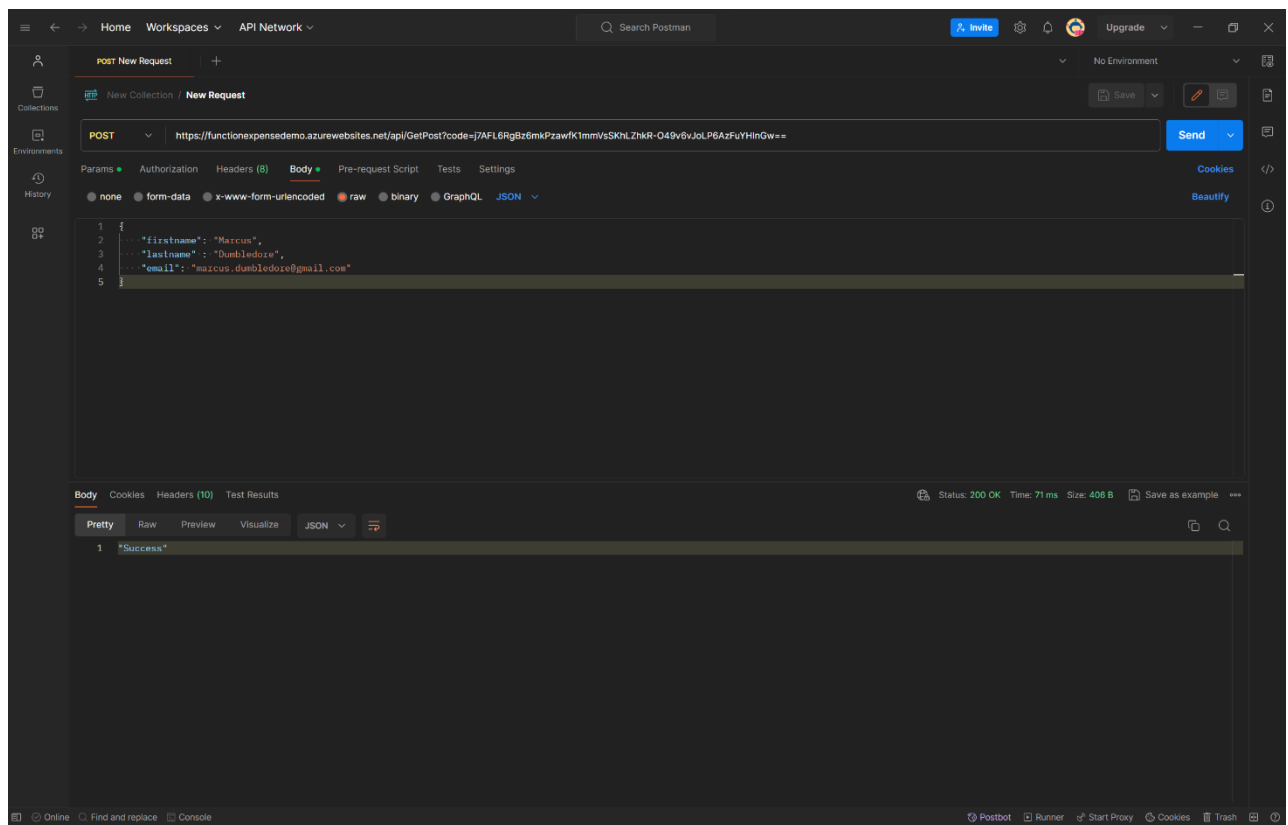
The screenshot shows the Visual Studio Code editor with the `GetPost.cs` file open. The code is a C# class `GetPost` with a static method `Run` that processes an HTTP request. The method logs the first name, last name, and email of the user from the request body and returns a success response. The `Output` window at the bottom shows the build and publish process, indicating that the function was successfully deployed to the Azure Functions environment.

```
1 using System.Linq;
2 using System.Net;
3 using System.Net.Http;
4 using System.Threading.Tasks;
5 using Microsoft.Azure.WebJobs.Extensions.Http;
6 using Microsoft.Azure.WebJobs.Host;
7
8 namespace DemoAzureFunction
9 {
10     [FunctionName("GetPost")]
11     public static async Task<HttpResponseMessage> Run([HttpTrigger(AuthorizationLevel.Function, "get", "post", Route = null)]HttpRequestMessage req, TraceWriter log)
12     {
13         log.Info("C# HTTP trigger function processed a request.");
14
15         dynamic data = await req.Content.ReadAsAsync<Object>();
16
17         log.Info(data?.firstname.ToString());
18         log.Info(data?.lastname.ToString());
19         log.Info(data?.email.ToString());
20
21         return req.CreateResponse(HttpStatusCode.OK, "Success");
22     }
23 }
```

Output:

```
2>Publishing C:\Users\Nicol\Desktop\Samitha's Course on Dynamics 365\11 20 12 23\DemoAzureFunction\DemoAzureFunction\obj\Release\net48\PubTmp\DemoAzureFunction - 20240113185235576.zip to https://function...
----- Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped -----
----- Publish: 1 succeeded, 0 failed, 0 skipped -----
----- Publish completed at 18:52 and took 11,783 seconds -----
Waiting for function app to be ready...
Function app is ready
```

Such function does nothing special; we are just tracing the values of first name, last name and email as acquired in the JSON we provide in Postman.



The screenshot shows the Postman application with a new POST request created. The URL is `https://functionexpensedemo.azurewebsites.net/api/GetPost?code=7AF16RgBz6mkPzawfK1mm/s5KhLZhKR-049v6xJolP6AzFuYHnGw==`. The request body is a JSON object containing the first name, last name, and email of a user. The response status is 200 OK, and the response body is `"Success"`.

POST New Request

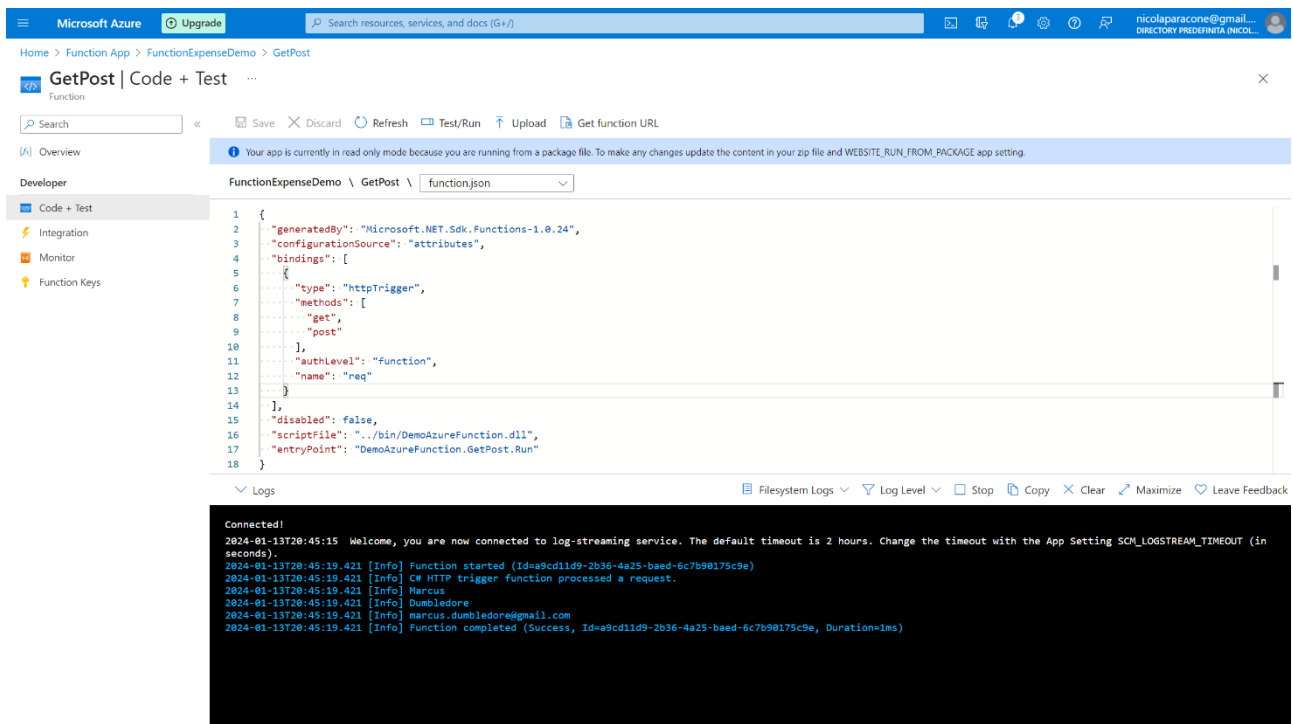
URL: `https://functionexpensedemo.azurewebsites.net/api/GetPost?code=7AF16RgBz6mkPzawfK1mm/s5KhLZhKR-049v6xJolP6AzFuYHnGw==`

Body (JSON):

```
1 {
2   "firstname": "Marcus",
3   "lastname": "Dumbledore",
4   "email": "marcus.dumbledore@gmail.com"
5 }
```

Response:

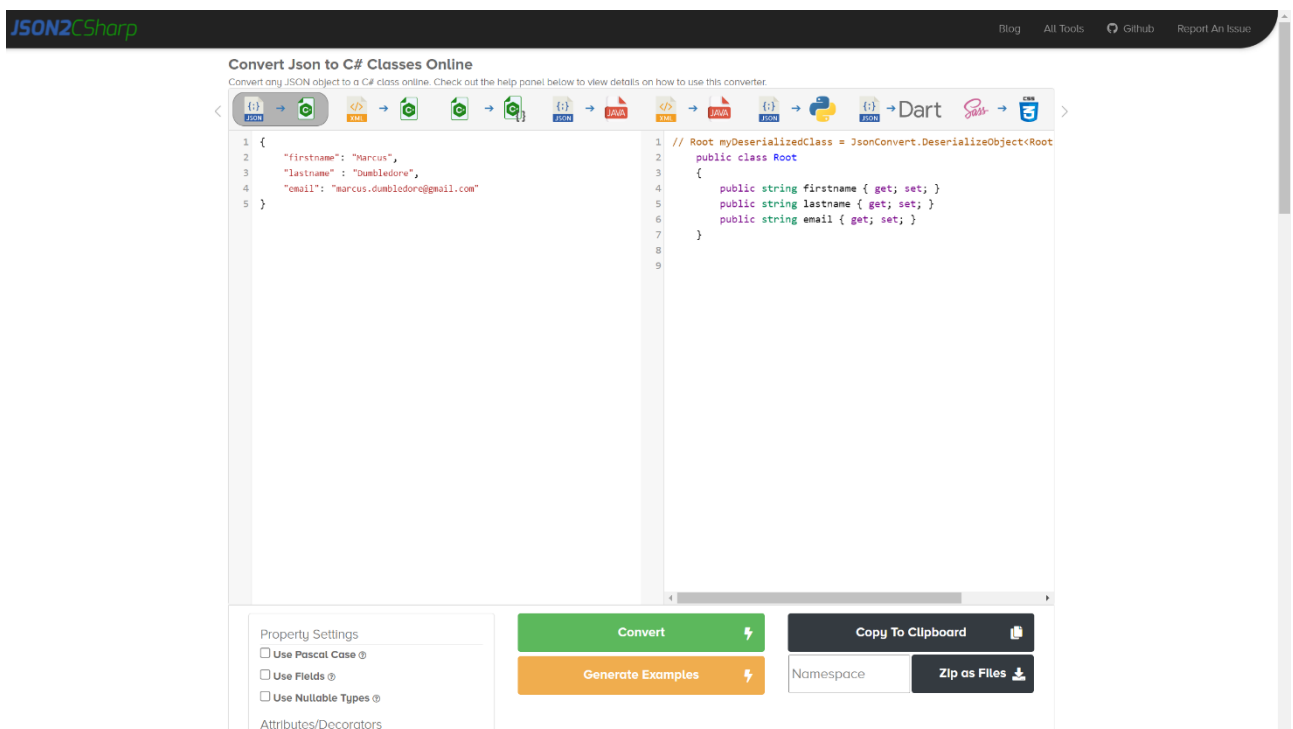
```
1 "Success"
```



In our example, the JSON code has only three key-value pairs and we are tracing their values by explicitly calling the Info method of the TraceWriter class.

6. Serialization and Deserialization

In order not to have to check each of these variables individually, we can create a **model class**. Navigate to <https://json2csharp.com> and paste the JSON we have in Postman.



In our example, we have a JSON object with three properties: `firstname`, `lastname`, and `email`. The corresponding C# class `Root` is a representation of this JSON structure, and it is used to deserialize the JSON data into an instance of the `Root` class using a JSON converter, such as the one provided by `Newtonsoft.Json` (`JsonConvert.DeserializeObject<Root>(myJsonResponse)`).

`Root` is the main class that represents the structure of your JSON data. It has three properties: `firstname`, `lastname`, and `email`, each of type `string`. The property names in the C# class match the names in the JSON object. This is important for proper deserialization.

When we use `JsonConvert.DeserializeObject<Root>(myJsonResponse)`, we are telling the JSON converter to take the JSON data (`myJsonResponse`) and map it to an instance of the `Root` class. The converter will match the property names in the JSON with the corresponding property names in the C# class and populate the object accordingly.

If we have the following JSON:

```
{
    "firstname": "Marcus",
    "lastname": "Dumbledore",
    "email": "marcus.dumbledore@gmail.com"
}
```

after deserialization, we would get an instance of the `Root` class with the properties set:

```
Root myDeserializedClass = JsonConvert.DeserializeObject<Root>(myJsonResponse);
```

Now, `myDeserializedClass` would have its `firstname`, `lastname`, and `email` properties set to `"Marcus"`, `"Dumbledore"`, and `"marcus.dumbledore@gmail.com"`, respectively.

Let us further explore the meaning of **serialization** and **deserialization**.

Serialization and deserialization are processes used to convert data between different formats, typically between an in-memory object representation and a format suitable for storage or transmission. In the context of C# and many other programming languages, this often involves converting objects to and from a serialized format, such as JSON or XML.

Serialization: it is the process of converting an object's state into a format (usually a string or binary data) that can be easily stored, transmitted, or reconstructed. This is commonly used when we want to save the state of an object to a file, send it over a network, or store it in a database. In C#, we can use serialization to convert an object to JSON, XML, binary, or other formats. The `JsonConvert` class in the `Newtonsoft.Json` library is commonly used for JSON serialization, and the `XmlSerializer` class is used for XML serialization.

Example of JSON serialization using `Newtonsoft.Json`:

```
using Newtonsoft.Json;

public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}

// Serialize a Person object to JSON
Person person = new Person { Name = "John", Age = 30 };
string json = JsonConvert.SerializeObject(person);
```

Deserialization: it is the process of reconstructing an object from its serialized form. It involves taking data in a specific format (e.g., JSON, XML) and converting it back into an object that can be used in the program. In C#, deserialization is often performed using the same libraries used for serialization. For example, we can use JsonConvert for JSON deserialization and XmlSerializer for XML deserialization.

Example of JSON deserialization using Newtonsoft.Json:

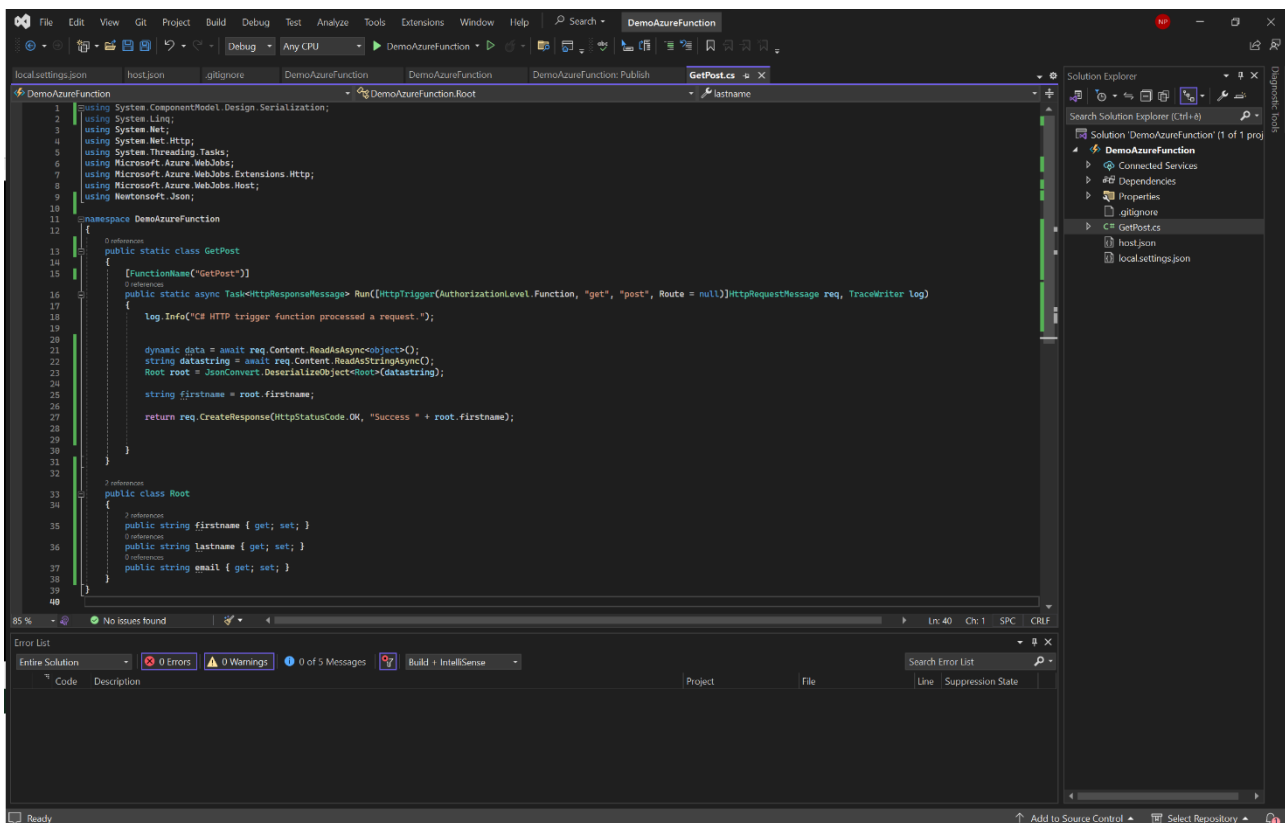
```
// Deserialize JSON to a Person object
```

```
Person deserializedPerson = JsonConvert.DeserializeObject<Person>(json);
```

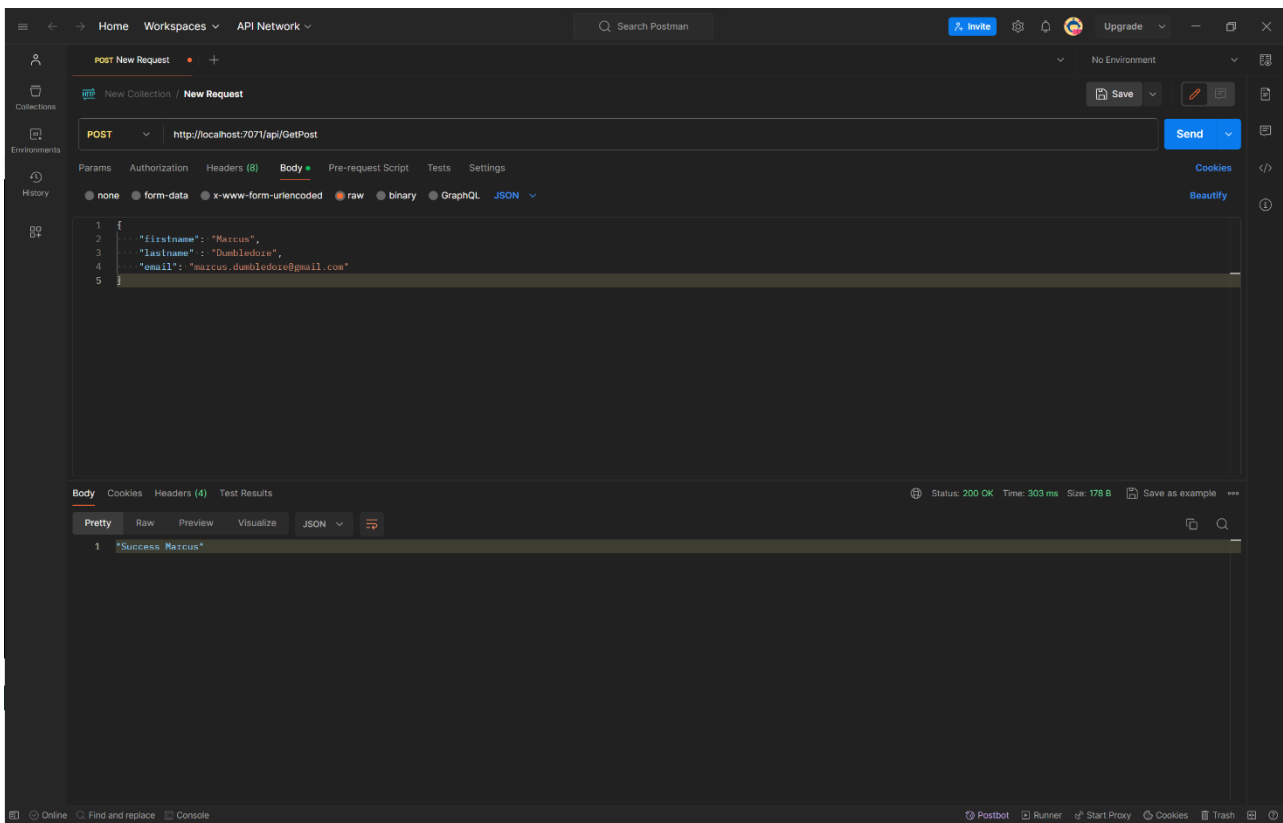
In this example, the json string, which was created by serializing a Person object earlier, is now deserialized back into a Person object.

Serialization and deserialization are crucial for various scenarios, such as saving and loading application state, exchanging data between different systems, or communicating over a network where the data needs to be represented in a compact and standardized format.

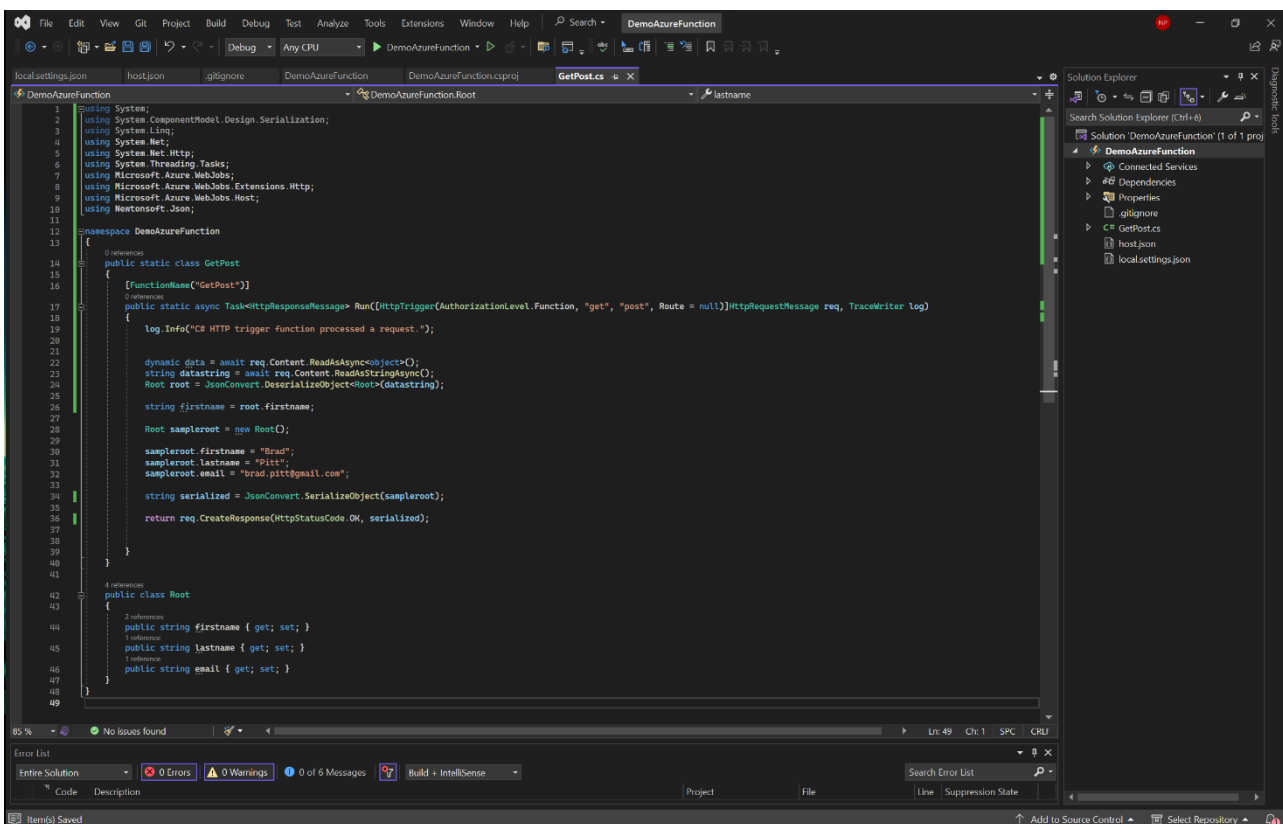
Let trigger from Postman the function in the following figure.



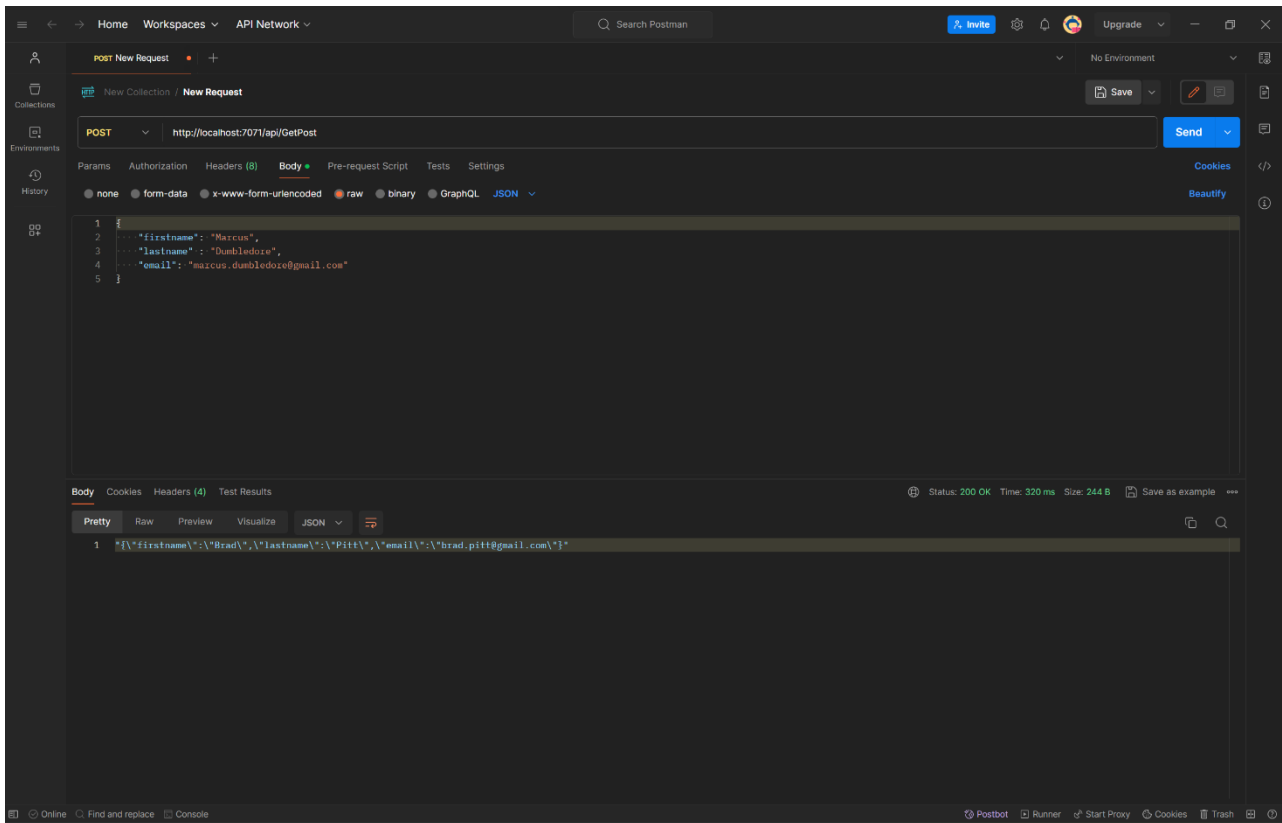
This Azure Function processes HTTP requests, expecting JSON data with at least a firstname property. It logs the incoming request, deserializes the JSON data into a Root object, extracts the firstname, and responds with a success message including the extracted firstname.



Let us see how to serialize data.



Once we trigger this function in Postman, we will obtain a JSON with key-value pairs derived from sampleroot.



A user receiving this JSON can then deserialize it as we previously did.

Suppose now that a user is sending us a JSON with data corresponding to multiple records.

```
[  {    \"firstname\": \"Marcus\",    \"lastname\": \"Dumbledore\",    \"email\": \"marcus.dumbledore@gmail.com\"  },  {    \"firstname\": \"Sam\",    \"lastname\": \"Snape\",    \"email\": \"sam.snape@gmail.com\"  },  {    \"firstname\": \"Jack\",    \"lastname\": \"Potter\",    \"email\": \"jack.potter@gmail.com\"  }]
```

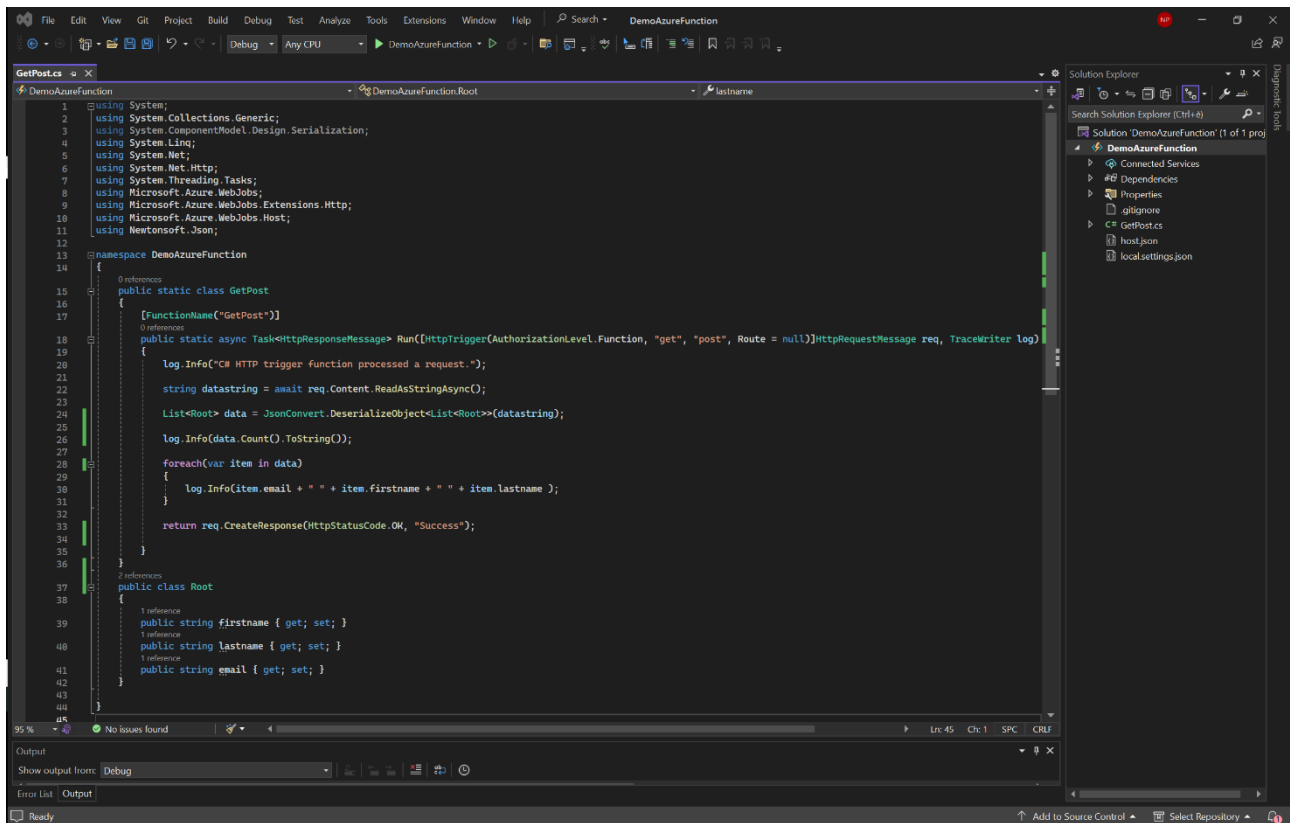
We can again use a JSON to C# converter to obtain the corresponding model class.

The screenshot shows the 'JSON2CSharp' website. The main heading is 'Convert Json to C# Classes Online'. Below it, a toolbar shows various programming languages: JSON, Kotlin, C#, Java, JavaScript, Python, Dart, and C#. The left pane contains a JSON array of three objects, each with 'firstname', 'lastname', and 'email' fields. The right pane shows the generated C# code, which includes a 'Root' class with a 'List' property and three public string properties for 'firstname', 'lastname', and 'email'. At the bottom, there are buttons for 'Convert', 'Generate Examples', 'Copy To Clipboard', and 'Zip as Files'. A 'Property Settings' panel on the left allows users to toggle options like 'Use Pascal Case', 'Use Fields', 'Use Nullable Types', and 'Attributes/Decorators'.

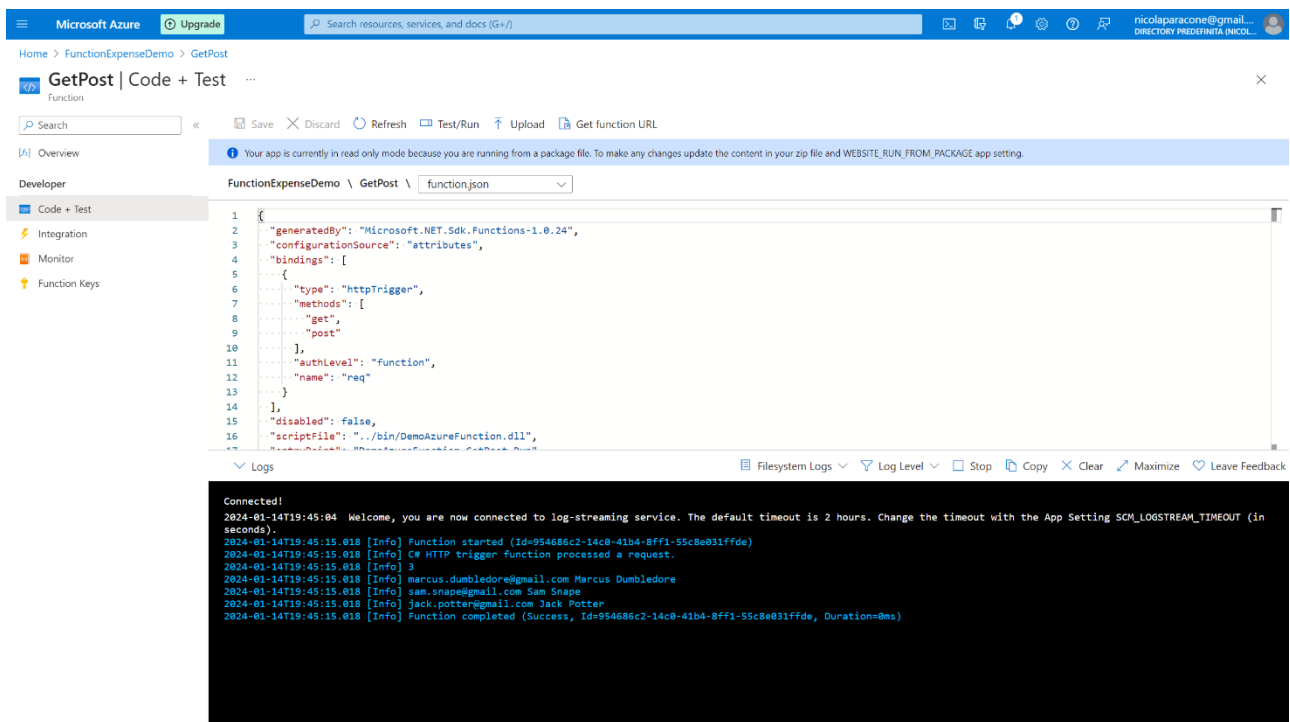
Moreover, if we are not sure about the validity of our JSON, we can validate it at <https://jsonlint.com>.

The screenshot shows the 'JSONLint' website, which is a 'Validator and Formatter'. It features a large text area for pasting JSON. Below the text area are buttons for 'Validate JSON', 'Clear', and 'Compress'. A green message bar at the bottom states 'JSON is valid!'. The website also includes an 'Ads by Google' banner at the top and a 'Send feedback' button at the bottom right.

No matter the number of records a user is sending us, each of them will be logged.

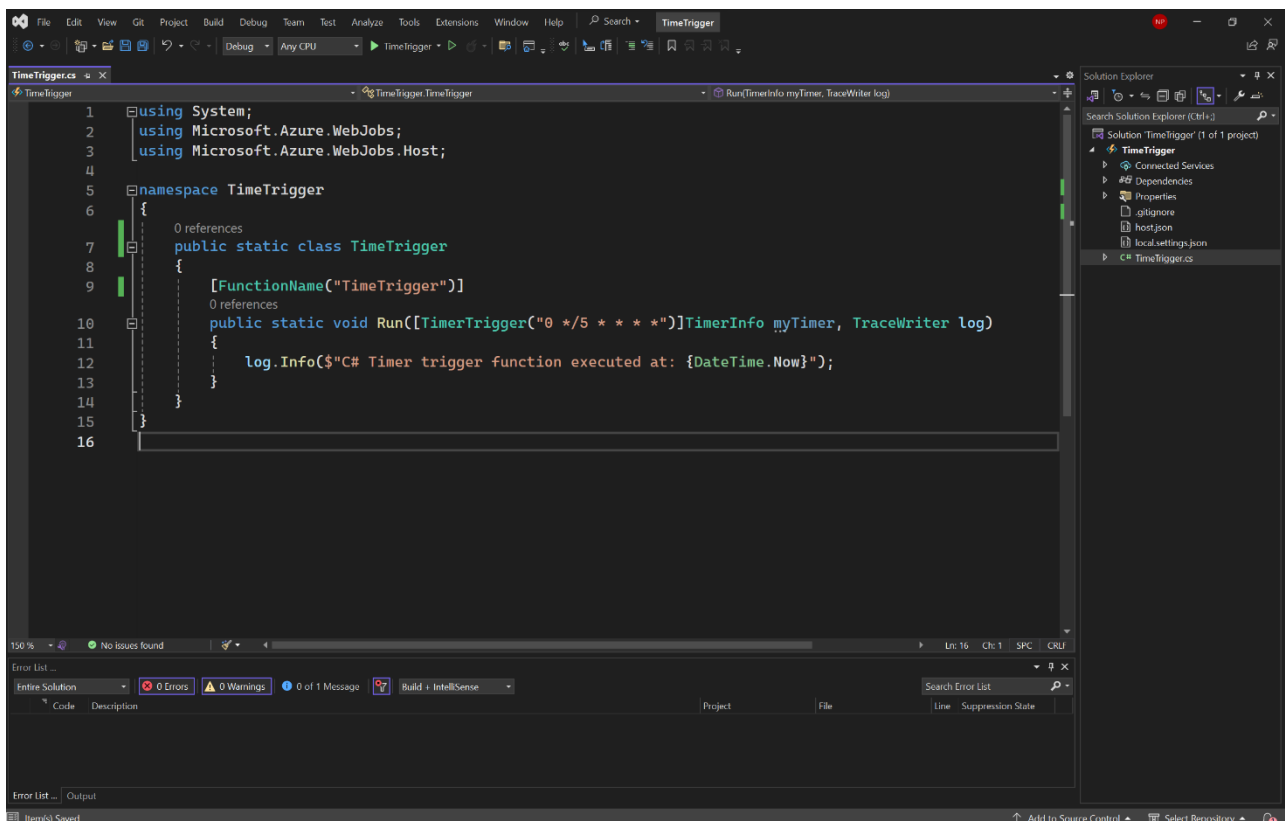
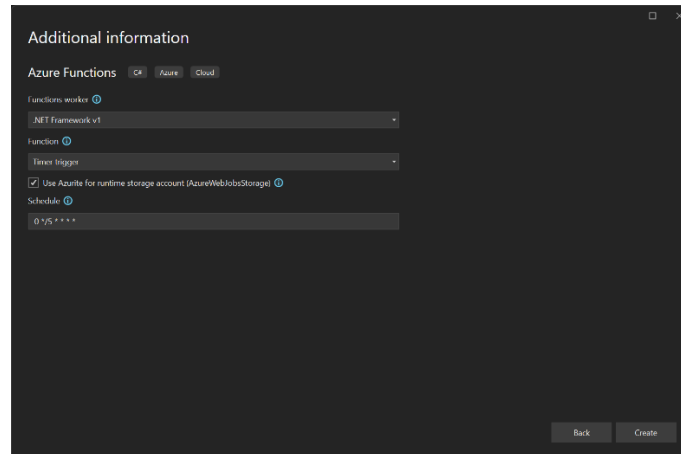


Publish this function in Azure and then trigger it from Postman, where have the JSON with three records. Once triggered from Postman, the logs output will be the following. Notice that we correctly visualize the records' data we expected.



7. Time-triggered functions

We now briefly explore time-triggered Azure functions. In this context, "time-triggered" refers to the ability of these functions to execute based on a specified schedule or recurring intervals. Instead of being triggered by an external event, such as an HTTP request or a message in a queue, time-triggered functions are initiated by a predefined timer. These functions are designed to automate recurring tasks, allowing developers to execute code at predetermined intervals without manual intervention. This capability is particularly useful for periodic maintenance, data processing, or any other scenario where regular, scheduled execution is required.



This Azure Function is designed to run on a time-based trigger, executing every 5 minutes, and logging information about its execution time using the specified TraceWriter.

Navigate to the following link for more instructions:

<https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-timer?tabs=python-v2%2Cisolated-process%2Cnodejs-v4&pivots=programming-language-csharp>

Notice in particular the following tables, which show some details on how to set the time trigger of the function.

Type	Example	When triggered
A specific value	0 5 * * * *	Once every hour of the day at minute 5 of each hour
All values (*)	0 * 5 * * *	At every minute in the hour, during hour 5
A range (- operator)	5-7 * * * * *	Three times a minute - at seconds 5 through 7 during every minute of every hour of each day
A set of values (, operator)	5,8,10 * * * * *	Three times a minute - at seconds 5, 8, and 10 during every minute of every hour of each day
An interval value (/ operator)	0 */5 * * * *	12 times an hour - at second 0 of every 5th minute of every hour of each day

Example	When triggered
0 */5 * * * *	once every five minutes
0 0 * * * *	once at the top of every hour
0 0 */2 * * * *	once every two hours
0 0 9-17 * * * *	once every hour from 9 AM to 5 PM
0 30 9 * * * *	at 9:30 AM every day
0 30 9 * * * 1-5	at 9:30 AM every weekday
0 30 9 * Jan Mon	at 9:30 AM every Monday in January